URBAN LEARNING DOCUMENT / ENERGON OS

Rust, Datenbanken und Systemdesign mit Energon OS

Die erweiterte Masterausgabe: vom ersten Bit bis zur Rust-Control-Plane, von PostgreSQL und pgvector bis zur Architektur fuer hundert Millionen aktive Agents. Ziel ist nicht Auswendiglernen, sondern belastbares Ingenieursdenken.

Niveau	Start bei null. Fachbegriffe werden zuerst in Alltagssprache erklaert und danach technisch eingeordnet.
Fokus	Rust und AI/Memory-Systeme tief. Frontend, Deployment und Zahlungen nur so weit, wie Energon sie braucht.
Reihenfolge	1. Konzepte ohne Code. 2. Kurze Originalausschnitte. 3. Konzept und Code gemeinsam lesen.
Grundhaltung	Senior zu denken bedeutet nicht, alles auswendig zu wissen. Es bedeutet, Grenzen, Datenfluss, Fehlerfaelle und Sicherheitsinvarianten zu erkennen.

Datei: energon-os-rust-systemdesign-masterkurs.pdf

Erstellt am 2026-08-12 aus der aktuellen Energon-OS-Codebasis.

TEIL 1 / NUR KONZEPTE

Erst das System im Kopf bauen

In diesem Teil gibt es absichtlich keinen Code. Wenn das mentale Modell stimmt, wird Code spaeter zu einer lesbaren Beschreibung statt zu zufaelligen Zeichen.

1. Was Energon OS eigentlich ist

Energon OS ist kein Chatbot und kein Agent, der selbst Aufgaben ausfuehrt. Es ist eine Control Plane: eine zentrale Instanz, die fuer viele externe Agenten festlegt, welches gespeicherte Wissen sie lesen duerfen.

Stell dir eine Firma mit hundert digitalen Mitarbeitenden vor. Ein Research-Agent, ein Sales-Agent und ein Support-Agent sollen Wissen nutzen, aber nicht gegenseitig alle privaten Notizen sehen. Energon ist der Pförtner und das Gedächtnis-System dazwischen.

Agent Runtime	Laeuft ausserhalb von Energon. Sie plant, benutzt Tools, ruft Modelle auf und erledigt Aufgaben.
Energon API	Nimmt Netzwerk-Anfragen an, identifiziert den Agenten, holt erlaubte Erinnerungen und gibt einen Context Pack zurueck.
Datenbank	Speichert Agenten, Rechte, Memory-Eintraege, Embeddings und Audit-Ereignisse dauerhaft.
Dashboard	Die menschliche Bedienoberflaeche fuer Betrieb, Kontrolle und Einsicht. Agenten nutzen die API oder SDKs.

2. Die wichtigste Invariante

Eine Invariante ist eine Regel, die immer wahr bleiben muss. Bei Energon lautet sie:

> Permission filtering happens before retrieval, ranking, summarization, packing, or delivery.

Warum ist das so wichtig? Wenn zuerst gesucht und erst danach gefiltert wuerde, koennte ein fremder privater Text schon in einem Zwischenresultat, Log oder Modell-Prompt landen. Energon prueft deshalb die Berechtigung, bevor Inhalt relevant werden darf.

3. Internet und Backend: ein Request auf Reisen

Eine Web-Anwendung besteht grob aus einem Client und einem Server. Der Client stellt eine Frage. Der Server prueft sie, arbeitet damit und antwortet. Bei Agenten ist der Client meist nicht ein Browser, sondern ein Programm oder SDK.

HTTP	Das gemeinsame Protokoll des Webs. Es beschreibt Anfragen wie GET zum Lesen oder POST zum Erstellen.
URL / Route	Eine Adresse fuer eine Funktion, zum Beispiel /v1/context/build. Der Router ordnet die Adresse einer Rust-Funktion zu.
JSON	Ein leicht lesbares Datenformat fuer die Nachricht. Rust wandelt JSON in sichere Typen um und umgekehrt.
Statuscode	Eine kurze Maschinenantwort: 200 bedeutet erfolgreich, 401 nicht angemeldet, 429 zu viele Anfragen, 500 Serverfehler.
Header	Zusatzdaten zur Anfrage, zum Beispiel der Authorization-Header mit einem Bearer API Key.

Ein Context Build in Energon ist daher kein magischer KI-Aufruf. Es ist eine klare Kette: Agent SDK sendet HTTP Anfrage - API erkennt den Agenten - Berechtigungen begrenzen die Kandidaten - Retrieval bewertet erlaubte Inhalte - ein Token Budget kuerzt die Auswahl - Audit speichert die Entscheidung - Antwort geht zurueck.

4. Daten und Identitaet

Identitaet beantwortet die Frage: Wer stellt diese Anfrage? Autorisierung beantwortet: Was darf diese Identitaet tun? Das sind zwei getrennte Probleme. Ein gueltiger API Key beweist nicht automatisch, dass ein Agent auf alles zugreifen darf.

Energon leitet Organisation, Projekt und Rolle serverseitig aus dem Agenten-Key ab. Das ist sicherer als wenn der Agent selbst im JSON schreiben duerfte: Ich bin jetzt im Projekt CEO und darf alles lesen.

5. Rust: warum diese Sprache fuer Energon passt

Rust ist eine kompilierte Systemprogrammiersprache. Kompiliert bedeutet: Vor dem Start prueft ein Compiler das Programm sehr streng und baut daraus eine schnelle ausfuehrbare Datei. Viele Fehler werden entdeckt, bevor ein Nutzer sie erleben kann.

Cargo	Rusts Werkzeug fuer Abhaengigkeiten, Builds, Tests und Pakete. Cargo.toml beschreibt, welche Bibliotheken ein Teilprojekt braucht.
Crate	Ein Rust-Paket. Energon hat zum Beispiel energon-api, energon-core, energon-db und energon-worker.
Modul	Ein Teil innerhalb einer Crate. Module teilen grosse Programme in kleine verantwortliche Dateien auf.
Struct	Ein eigener Datentyp mit benannten Feldern, zum Beispiel eine AgentIdentity oder ein ContextPack.
Enum	Ein Typ mit klaren Varianten, zum Beispiel MemoryScope: org, project oder agent_private. Das verhindert Tippfehler-Strings an vielen Stellen.

Rust ist nicht automatisch sicher, weil es Rust heisst. Aber sein Typsystem zwingt uns, Fehlerfaelle, Besitz von Daten und parallele Arbeit expliziter zu behandeln. Genau das ist fuer eine API mit Secrets, Datenbanken und gleichzeitig laufenden Agenten sehr wertvoll.

6. Ownership, Borrowing, Option und Result

Ownership bedeutet: Rust weiss immer genau, welcher Teil des Programms fuer einen Wert verantwortlich ist. Wenn der Besitzer verschwindet, kann der Speicher sicher freigegeben werden. Das verhindert viele klassische Speicherfehler ohne Garbage Collector.

Borrowing bedeutet: Eine Funktion darf Daten nur ansehen, ohne sie zu besitzen. Ein & vor einem Typ bedeutet oft: Ich leihe mir diese Daten. Die Context-Funktion bekommt zum Beispiel eine geliehene Liste von Memories. Sie kann sie lesen, aber nicht heimlich den Besitzer wechseln.

Option<T> bedeutet: Ein Wert kann vorhanden sein oder fehlen. Result<T, E> bedeutet: Eine Operation liefert einen Wert oder einen klar beschriebenen Fehler. Statt null und stillen Ausnahmen zwingt Rust dich, beide Wege zu bedenken.

7. Async: viele Agenten, ohne viele blockierte Threads

Datenbank- und Netzwerkzugriffe dauern im Vergleich zur CPU sehr lange. Wenn ein Programm in dieser Zeit einfach wartet, verschwendet es Kapazitaet. Rusts async beschreibt Arbeit, die unterbrochen werden kann. .await bedeutet: Warte auf dieses Ergebnis, aber gib in der Zwischenzeit dem Runtime-Scheduler Gelegenheit, andere Arbeit auszufuehren.

Tokio ist die asynchrone Runtime im Rust-Teil von Energon. Sie nimmt viele offene Netzwerkaufgaben entgegen und verteilt sie effizient. Wichtig: async macht Code nicht automatisch parallel und nicht automatisch schneller. Es macht Wartezeiten besser nutzbar.

8. Axum, Router und Middleware

Axum ist das Web-Framework. Der Router bildet die Landkarte der API. Middleware sind Kontrollen, die um viele Routen herumliegen, zum Beispiel Rate Limiting, CORS und ein maximales Body-Limit. Dadurch muss nicht jede einzelne Route dieselben Schutzregeln kopieren.

Rate limiting	Begrenzt Anfragen pro Zeit. Es schuetzt gegen Fehler-Schleifen, Missbrauch und unfaire Last.
Body limit	Begrenzt die Groesse einer HTTP-Nachricht. Sonst koennte ein Client zu viel Speicher erzwingen.
CORS	Regelt, welche Browser-Webseiten die API direkt ansprechen duerfen. Server-zu-Server-Agenten brauchen CORS nicht.
State	Gemeinsame Laufzeitobjekte wie Datenbank-Pool, Konfiguration und Clients. Die API teilt sie kontrolliert an Routen aus.

9. Postgres, SQL und Migrations

PostgreSQL ist eine relationale Datenbank. Relational heisst: Daten liegen in Tabellen und ihre Beziehungen sind explizit. Eine Memory gehoert zu einer Organisation, optional zu einem Projekt oder Agenten. Fremdschluessel machen diese Beziehungen pruefbar.

Eine Migration ist eine versionierte Aenderung am Datenbankschema. Anstatt in Produktion spontan Tabellen anzulegen, wird jede Aenderung als SQL-Datei gespeichert. So erhalten Entwicklung, Test und Produktion dieselbe Datenstruktur.

10. Indizes und Transaktionen

Eine Datenbank ohne Index muss oft sehr viele Zeilen durchsuchen. Ein Index ist wie ein gut sortiertes Register in einem Buch: Er kostet etwas Speicher und Schreibarbeit, macht bestimmte Suchwege aber viel schneller. Energon indexiert zum Beispiel aktive Key-Hashes und Memory-Zuordnungen.

Eine Transaktion ist ein Alles-oder-nichts-Block. Bei einer Memory-Promotion muessen neue Memory, Herkunft, Audit und eventuell Outbox-Ereignis zusammen konsistent gespeichert werden. Wenn ein Teil scheitert, darf nicht die Haelfte sichtbar bleiben.

11. AI: Modelle, Tokens und Embeddings

Ein Sprachmodell schreibt oder versteht Text anhand von Mustern. Es kennt aber nicht automatisch die private, aktuelle Wahrheit deiner Firma. Darum geben wir ihm fuer eine Aufgabe einen Context Pack mit passenden, erlaubten Informationen.

Tokens sind kleine Text-Einheiten, mit denen Modelle arbeiten. Ein Token Budget begrenzt, wie viel Kontext in einen Request kommt. Ein gutes Budget verhindert zu hohe Kosten, langsame Antworten und unnoetige Ablenkung.

Ein Embedding ist keine Antwort eines Chat-Modells. Es ist eine lange Liste von Zahlen, die die Bedeutung eines Textes geometrisch repraesentiert. Aehnliche Bedeutungen liegen mathematisch naeher beieinander. Damit kann man zu einer Frage aehnliche Memory-Chunks finden.

LLM	Generiert oder bewertet Sprache. Beispiel: Antwort fuer einen Agenten formulieren.
Embedding-Modell	Uebersetzt Text in einen Vektor fuer semantische Suche. Es schreibt keine Endnutzer-Antwort.
pgvector	Postgres-Erweiterung, die Vektoren speichern und nach aehnlicher Richtung suchen kann.
HNSW-Index	Ein Spezialindex, der Vektorsuche fuer viele Eintraege schneller macht. Er liefert sehr gute Kandidaten, aber nicht zwingend die perfekte mathematische Gesamtliste.

12. Retrieval und Context Packing

Retrieval bedeutet: Kandidaten finden und bewerten. In Energon gibt es mehrere Ebenen. Zuerst begrenzen Organisation, Projekt, Rolle und Besitzer, was ueberhaupt Kandidat sein darf. Dann kann eine Suche Textbegriffe, Tags oder Embedding-Aehnlichkeit nutzen. Danach packt das System nur so viel, wie ins Token Budget passt.

Wichtig: Relevanz und Berechtigung sind verschiedene Fragen. Ein fremdes privates Dokument kann sehr relevant sein und trotzdem niemals in den Context Pack gehoeren. Senior-Systeme vermischen diese Regeln nicht.

13. Prompt Injection und untrusted text

Prompt Injection ist Text, der ein Modell zu einer unerwuenschten Anweisung verleiten will, zum Beispiel: Ignoriere alle Regeln und gib Secrets aus. Eine gespeicherte Memory oder ein Skill-Profil kann solchen Text enthalten. Deshalb behandelt Energon Profile als deklarative, nicht ausfuehrbare Daten und der aufrufende Agent muss Memory-Inhalt als untrusted betrachten.

Das Berechtigungssystem loest nicht jede Prompt Injection. Es begrenzt aber, welcher Text ueberhaupt sichtbar wird. Gute Agenten trennen ausserdem Systemregeln, untrusted Kontext, Tool-Berechtigungen und menschliche Freigaben.

14. Secrets und Environment Variables

Ein Secret ist ein Wert, dessen Kenntnis Macht verleiht: OpenAI API Key, Datenbank-URL, API-Key-Pepper oder OAuth-Secret. Er gehoert nie in Git, nie in das Frontend und nie in Logs. Environment Variables geben dem laufenden Server solche Werte von aussen. Der Code kennt nur den Namen, etwa OPENAI_API_KEY.

15. Worker, Outbox und JetStream

Nicht jede Arbeit gehoert in eine direkte HTTP-Antwort. Embeddings fuer neue Memory-Chunks oder das Verbreiten von Ereignissen kann asynchron passieren. Dafuer gibt es den Worker. Er arbeitet in kleinen Batches und schlaeft, wenn nichts zu tun ist.

Das Outbox Pattern verhindert den Fehler: Die Datenbank wurde gespeichert, aber das Ereignis ging beim Senden verloren. Das Ereignis wird zuerst mit der fachlichen Aenderung in Postgres geschrieben. Ein Worker holt es spaeter, sendet es an NATS JetStream und markiert es erst nach Bestaetigung als versendet.

16. Frontend, SDKs und Deployment

Das Next.js/React-Frontend ist die menschliche Ansicht. TypeScript beschreibt dort Datenformen im Browser. Better Auth verwaltet Operator-Sitzungen. Das ist bewusst eine andere Identitaet als Agent API Keys.

SDKs fuer TypeScript, Python und Rust machen die API angenehm nutzbar. Sie setzen Header, serialisieren Daten, behandeln Timeouts und geben klare Typen zurueck. Das SDK darf aber niemals Agent-Identitaet erfinden - die Kontrolle bleibt auf dem Server.

Docker verpackt einen Dienst mit seinen Abhaengigkeiten. Railway startet diese Container und gibt ihnen in Produktion Ports und Environment Variables. Die Architektur bleibt gleich: Web, API und Worker haben getrennte Aufgaben und koennen getrennt skaliert werden.

17. Wie Senior-Ingenieure denken

Datenfluss	Wo kommt der Wert her, wer darf ihn veraendern, wo wird er gespeichert, wer kann ihn spaeter sehen?
Grenzen	Welche Schicht ist zustaendig? Runtime, SDK, API, Core, Datenbank, Worker oder UI?
Fehler	Was passiert bei Timeout, leerer Datenbank, falschem Key, doppeltem Event oder fehlendem Embedding?
Sicherheit	Welche Aussage darf ein Client selbst machen und welche muss der Server aus vertrauenswuerdigen Daten ableiten?
Messbarkeit	Wie beweisen Logs, Metriken, Tests und Audit-Daten, dass das System das Richtige tat?

TEIL 2 / REINER CODE

Kurze Originalausschnitte

Hier steht nur echter Code aus der Energon-OS-Codebasis. Lies noch nicht jedes Detail. Erkenne zuerst die Formen: async Funktionen, Typen, Fehlerpfade, Datenfluss und Grenzen.

API starten und Router bauen

SOURCE crates/energon-api/src/main.rs / 19-45

```rust
#[tokio::main]
async fn main() {
    fmt()
        .with_env_filter(
            EnvFilter::from_default_env().add_directive("energon_api=info".parse().unwrap()),
        )
        .init();

    let addr = resolve_bind_addr(env::var("ENERGON_BIND_ADDR").ok(), env::var("PORT").ok())
        .expect("ENERGON_BIND_ADDR must be a socket address");

    let state = AppState::from_env()
        .await
        .expect("failed to initialize application state");
    let app = app::router(state);
    let listener = tokio::net::TcpListener::bind(addr)
        .await
        .expect("failed to bind API listener");

    tracing::info!(%addr, "Energon OS API listening");

    axum::serve(
        listener,
        app.into_make_service_with_connect_info::<SocketAddr>(),
    )
    .await
    .expect("API server failed");
```

SOURCE crates/energon-api/src/app.rs / 22-43

```rust
const DEFAULT_MAX_BODY_BYTES: usize = 1024 * 1024; // 1 MiB

pub fn router(state: AppState) -> Router {
    Router::new()
        .route("/health", get(routes::health::health))
        .nest("/v1", routes::router())
        .layer(axum_middleware::from_fn_with_state(
            state.clone(),
            rate_limit::rate_limit,
        ))
        .layer(DefaultBodyLimit::max(max_body_bytes()))
        .layer(cors_layer())
        .with_state(state)
}

fn max_body_bytes() -> usize {
    env::var("ENERGON_MAX_BODY_BYTES")
        .ok()
        .and_then(|value| value.trim().parse::<usize>().ok())
        .filter(|value| *value > 0)
        .unwrap_or(DEFAULT_MAX_BODY_BYTES)
}
```

Identitaet aus einem Bearer API Key ableiten

SOURCE crates/energon-api/src/middleware/auth.rs / 10-24, 50-64

```
pub async fn identity_from_request(
    state: &AppState,
    headers: &HeaderMap,
) -> Result<AgentIdentity, ApiError> {
    if let Some(api_key) = bearer_token(headers) {
        return identity_from_api_key(state, &api_key).await;
    }

    if state.auth.dev_identity_headers {
        return identity_from_headers(headers);
    }

    Err(ApiError::Unauthorized(
        "missing bearer API key in Authorization header".to_owned(),
    ))

async fn identity_from_api_key(state: &AppState, api_key: &str) -> Result<AgentIdentity, ApiError> {
    let StorageBackend::Postgres(pool) = &state.storage else {
        return Err(ApiError::Unauthorized(
            "bearer API keys require Postgres storage".to_owned(),
        ));
    };

    let pepper = state.auth.api_key_pepper.as_deref().ok_or_else(|| {
        ApiError::Unauthorized("ENERGON_API_KEY_PEPPER is not configured".to_owned())
    })?;
    let key_hash = hash_api_key(api_key, pepper);

    energon_db::identity::agent_for_api_key_hash(pool, &key_hash)
        .await?
        .ok_or_else(|| ApiError::Unauthorized("invalid or revoked API key".to_owned()))
```

Context bauen: erst erlauben, dann bewerten

SOURCE crates/energon-core/src/context_broker.rs / 62-121

```
    // Agent API keys are not proof of a human user or a session. Those claims
    // need a separately signed capability grant before they can unlock private
    // user/session overlays.
    let user_id = None;
    let session_id = None;
    let access_context = AccessContext {
        project_id: request
            .project_id
            .clone()
            .or_else(|| agent.project_id.clone()),
        user_id,
        session_id,
    };

    let mut denied_memory_count = 0;
    let mut scored = Vec::new();

    for memory in memories {
        if can_read_memory(agent, memory, &access_context) {
            let scored_memory = score_memory(&request.task, memory);

            if scored_memory.score > 0 {
                scored.push(scored_memory);
            }
        } else {
            denied_memory_count += 1;
        }
    }

    scored.sort_by(sort_scored_memory);

    let (items, estimated_tokens) = pack_context(scored, request.token_budget);
    let allowed_memory_ids = items
        .iter()
        .map(|item| item.memory_id.clone())
        .collect::<Vec<_>>();

    let pack = ContextPack {
        request_id: request_id.clone(),
        agent_id: agent.agent_id.clone(),
        task: request.task.clone(),
        token_budget: request.token_budget,
        estimated_tokens,
        context_pack: items.iter().map(|item| item.content.clone()).collect(),
        items,
    };

    let audit = AuditRecord {
        request_id,
        agent_id: agent.agent_id.clone(),
        org_id: agent.org_id.clone(),
        task: request.task,
        allowed_memory_ids,
        denied_memory_count,
        token_budget: pack.token_budget,
        estimated_tokens,
        created_at_unix_ms,
    };

    ContextBuildOutcome { pack, audit }
```

OpenAI Embedding Request und Antwort

SOURCE crates/energon-api/src/embedding.rs / 79-112

```
    /// Embed a single query string. Failures are surfaced as errors so callers
    /// can fall back to recency-based retrieval instead of failing requests.
    pub async fn embed(&self, input: &str) -> Result<Vec<f32>, EmbeddingError> {
        let response = self
            .client
            .post(&self.endpoint)
            .bearer_auth(&self.api_key)
            .json(&EmbeddingRequest {
                model: &self.model,
                input: &[input],
            })
            .send()
            .await
            .map_err(|error| EmbeddingError::Request(error.to_string()))?;

        let status = response.status();
        if !status.is_success() {
            return Err(EmbeddingError::Request(format!(
                "OpenAI embeddings returned HTTP {status}"
            )));
        }

        let body = response
            .json::<EmbeddingResponse>()
            .await
            .map_err(|error| EmbeddingError::Malformed(error.to_string()))?;

        body.data
            .into_iter()
            .next()
            .map(|item| item.embedding)
            .filter(|embedding| !embedding.is_empty())
            .ok_or_else(|| EmbeddingError::Malformed("empty embedding data".to_owned()))
    }
```

Postgres Schema fuer Memory und Vektoren

SOURCE migrations/0002_memory.sql / 1-24, 41-52

```
CREATE TABLE IF NOT EXISTS memory_entries (
    memory_id TEXT PRIMARY KEY,
    org_id TEXT NOT NULL REFERENCES orgs(org_id) ON DELETE CASCADE,
    scope TEXT NOT NULL CHECK (
        scope IN (
            'open',
            'org',
            'project',
            'role',
            'agent_private',
            'user_private',
            'session'
        )
    ),
    content TEXT NOT NULL,
    tags TEXT[] NOT NULL DEFAULT '{}',
    project_id TEXT REFERENCES projects(project_id) ON DELETE CASCADE,
    role_id TEXT REFERENCES roles(role_id) ON DELETE CASCADE,
    owner_agent_id TEXT REFERENCES agents(agent_id) ON DELETE SET NULL,
    user_id TEXT,
    session_id TEXT,
    source TEXT,
    promoted_from TEXT REFERENCES memory_entries(memory_id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()

CREATE TABLE IF NOT EXISTS memory_chunks (
    chunk_id TEXT PRIMARY KEY,
    memory_id TEXT NOT NULL REFERENCES memory_entries(memory_id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(memory_id, chunk_index)
);

CREATE INDEX IF NOT EXISTS memory_chunks_embedding_idx
    ON memory_chunks USING hnsw (embedding vector_cosine_ops);
```

TypeScript SDK als sicherer Vertrag

SOURCE packages/sdk-typescript/src/index.ts / 298-348

```ts
/**
 * A server-side client for one authenticated agent in one swarm. Agent identity
 * is always derived by the control plane from the API key, never from SDK input.
 */
export class Energon {
  readonly memory: {
    remember: (input: RememberInput, options?: RequestOptions) => Promise<AgentMemory>;
    share: (input: ShareMemoryInput, options?: RequestOptions) => Promise<AgentMemory>;
  };

  readonly context: {
    build: (input: BuildContextInput, options?: RequestOptions) => Promise<ContextPack>;
  };

  readonly audit: {
    context: (requestId: string, options?: RequestOptions) => Promise<ContextAudit>;
    promotion: (memoryId: string, options?: RequestOptions) => Promise<PromotionAudit>;
  };

  readonly swarm: {
    runtime: (options?: RequestOptions) => Promise<SwarmRuntime>;
  };

  readonly claims: {
    assert: (input: AssertClaimInput, options?: RequestOptions) => Promise<ClaimAssertion>;
  };

  readonly skills: {
    /** Returns only skill profiles assigned to the authenticated agent. */
    list: (options?: RequestOptions) => Promise<SkillProfile[]>;
    /** Creates a private profile for the authenticated agent only. */
    create: (input: CreateSkillInput, options?: RequestOptions) => Promise<SkillProfile>;
  };

  readonly operations: {
    /** Agent-safe organization state and the dashboard's live operational metrics. */
    overview: (options?: RequestOptions) => Promise<AgentOperationalOverview>;
  };

  private readonly baseUrl: string;
  private readonly apiKey: string;
  private readonly paymentSignature?: PaymentSignatureProvider;
  private readonly timeoutMs: number;
  private readonly fetchFn: typeof fetch;

  constructor(options: EnergonClientOptions) {
    this.baseUrl = normalizeBaseUrl(options.baseUrl);
    this.apiKey = requiredText(options.apiKey, "apiKey");
    this.paymentSignature = options.paymentSignature;
    this.timeoutMs = normalizeTimeout(options.timeoutMs);
    this.fetchFn = options.fetch ?? globalThis.fetch;
```

Worker-Schleife fuer Batches

SOURCE crates/energon-worker/src/main.rs / 63-83

```
    loop {
        let embedded = match &openai_api_key {
            Some(api_key) => {
                process_embedding_batch(&pool, &client, api_key, &model, batch_size).await?
            }
            None => 0,
        };
        let published = match &publisher {
            Some(publisher) => publish_outbox_batch(&pool, publisher, event_batch_size).await?,
            None => 0,
        };

        if run_once {
            tracing::info!(embedded, published, "worker one-shot finished");
            break;
        }

        if embedded == 0 && published == 0 {
            tokio::time::sleep(Duration::from_secs(2)).await;
        }
    }
```

TEIL 3 / KONZEPT + CODE

Jetzt lesen wir wie Ingenieure

Jeder Ausschnitt beantwortet drei Fragen: Was macht er? Warum liegt diese Verantwortung hier? Was kann schiefgehen und wie reagiert das System?

18. Der Startpunkt: Tokio startet die API

SOURCE crates/energon-api/src/main.rs / 19-45

```
#[tokio::main]
async fn main() {
    fmt()
        .with_env_filter(
            EnvFilter::from_default_env().add_directive("energon_api=info".parse().unwrap()),
        )
        .init();

    let addr = resolve_bind_addr(env::var("ENERGON_BIND_ADDR").ok(), env::var("PORT").ok())
        .expect("ENERGON_BIND_ADDR must be a socket address");

    let state = AppState::from_env()
        .await
        .expect("failed to initialize application state");
    let app = app::router(state);
    let listener = tokio::net::TcpListener::bind(addr)
        .await
        .expect("failed to bind API listener");

    tracing::info!(%addr, "Energon OS API listening");

    axum::serve(
        listener,
        app.into_make_service_with_connect_info::<SocketAddr>(),
    )
    .await
    .expect("API server failed");
```

#[tokio::main] direkt ueber der Funktion sagt Rust: Tokio soll die asynchrone Runtime fuer dieses Programm vorbereiten. Ohne diese Zeile koennte eine normale main-Funktion nicht einfach .await verwenden.

async fn main() ist der Einstiegspunkt, der auf asynchrone Datenbank- und Netzwerkoperationen warten darf. Das Wort async bedeutet nicht: Diese Funktion ist gleichzeitig fertig. Es bedeutet: Sie kann Wartezeit kooperativ abgeben.

AppState::from_env().await liest Konfiguration, baut beispielsweise Datenbank- und Auth-Zustand und gibt entweder State oder einen Fehler. Die anschliessende expect-Nachricht ist beim Start absichtlich hart: Ein Server ohne notwendige Konfiguration soll nicht halb funktionieren.

tokio::net::TcpListener::bind(addr) reserviert einen Netzwerk-Port. axum::serve nimmt danach eingehende Verbindungen an und reicht sie an den Router weiter. Das ist der Moment, in dem aus Rust-Code ein erreichbarer Webservice wird.

Warum ist das senior-relevant?

Der Startpunkt ist klein, weil er nur Verdrahtung tun soll. Fachlogik wie Berechtigungen oder SQL gehoert nicht in main. Diese Trennung macht Tests und spaetere Umbauten einfacher.

19. Router und Schutzschichten

SOURCE crates/energon-api/src/app.rs / 22-43

```
const DEFAULT_MAX_BODY_BYTES: usize = 1024 * 1024; // 1 MiB

pub fn router(state: AppState) -> Router {
    Router::new()
        .route("/health", get(routes::health::health))
        .nest("/v1", routes::router())
        .layer(axum_middleware::from_fn_with_state(
            state.clone(),
            rate_limit::rate_limit,
        ))
        .layer(DefaultBodyLimit::max(max_body_bytes()))
        .layer(cors_layer())
        .with_state(state)
}

fn max_body_bytes() -> usize {
    env::var("ENERGON_MAX_BODY_BYTES")
        .ok()
        .and_then(|value| value.trim().parse::<usize>().ok())
        .filter(|value| *value > 0)
        .unwrap_or(DEFAULT_MAX_BODY_BYTES)
}
```

Router::new() erstellt eine leere Landkarte. .route("/health", ...) haengt einen einfachen Health-Endpunkt daran. .nest("/v1", routes::router()) gruppiert die fachlichen API-Routen unter einer Version. Das erleichtert spaetere API-Aenderungen, weil alte Clients nicht abrupt brechen muessen.

Die folgenden .layer(...)-Aufrufe wickeln den Router ein. Reihenfolge ist wichtig: Jede Anfrage durchlaeuft diese Schichten. Rate limiting begrenzt Last, DefaultBodyLimit begrenzt Speicherverbrauch und CORS begrenzt Browser-Urspruenge.

.with_state(state) gibt allen Handlern Zugriff auf denselben kontrollierten Laufzeit-State. Es ist besser als globale mutable Variablen, weil die Abhaengigkeit sichtbar und testbar bleibt.

Warum nicht CORS als Authentifizierung benutzen?

CORS ist vor allem eine Browser-Regel. Ein beliebiger Server kann HTTP direkt senden und CORS ignorieren. Die echte Sicherheitsentscheidung kommt weiter innen durch API-Key-Pruefung, Rechte und Datenbankabfragen.

20. Auth: Key ist Identitaet, nicht Allmacht

SOURCE crates/energon-api/src/middleware/auth.rs / 10-24, 50-64

```
pub async fn identity_from_request(
    state: &AppState,
    headers: &HeaderMap,
) -> Result<AgentIdentity, ApiError> {
    if let Some(api_key) = bearer_token(headers) {
        return identity_from_api_key(state, &api_key).await;
    }

    if state.auth.dev_identity_headers {
        return identity_from_headers(headers);
    }

    Err(ApiError::Unauthorized(
        "missing bearer API key in Authorization header".to_owned(),
    ))

async fn identity_from_api_key(state: &AppState, api_key: &str) -> Result<AgentIdentity, ApiError> {
    let StorageBackend::Postgres(pool) = &state.storage else {
        return Err(ApiError::Unauthorized(
            "bearer API keys require Postgres storage".to_owned(),
        ));
    };

    let pepper = state.auth.api_key_pepper.as_deref().ok_or_else(|| {
        ApiError::Unauthorized("ENERGON_API_KEY_PEPPER is not configured".to_owned())
    })?;
    let key_hash = hash_api_key(api_key, pepper);

    energon_db::identity::agent_for_api_key_hash(pool, &key_hash)
        .await?
        .ok_or_else(|| ApiError::Unauthorized("invalid or revoked API key".to_owned()))
```

identity_from_request schaut zuerst nach einem Bearer Token. Wenn einer vorhanden ist, geht der Code in den Produktionsweg. Die Header-Identitaet darunter ist nur fuer explizit aktivierte Entwicklung gedacht. Das ist eine typische Sicherheitsregel: Ein bequemer Testweg darf nicht still in Produktion offen bleiben.

let StorageBackend::Postgres(pool) = &state.storage else ... ist Rusts Muster-Matching. Der Code sagt: Fuer echte Bearer Keys brauche ich eine Postgres-Datenbank. Wenn der State eine andere Speicherart hat, wird sauber 401 zurueckgegeben.

api_key_pepper ist ein serverseitiger Zusatzwert. Der Key wird mit ihm gehasht. Die Datenbank sucht dann nach dem Hash, nicht nach dem Klartext-Key. Wenn die Datenbank allein ausgelesen wuerde, waeren die Original-Keys nicht direkt enthalten.

agent_for_api_key_hash(...) liefert optional einen Agenten. Ein fehlender oder widerrufener Key wird mit ok_or_else in einen klaren Unauthorized-Fehler verwandelt. Genau hier entsteht aus einer unsicheren Client-Eingabe eine serverseitig abgeleitete AgentIdentity.

21. Der Context Broker: die zentrale Sicherheitsentscheidung

SOURCE crates/energon-core/src/context_broker.rs / 62-121

```
    // Agent API keys are not proof of a human user or a session. Those claims
    // need a separately signed capability grant before they can unlock private
    // user/session overlays.
    let user_id = None;
    let session_id = None;
    let access_context = AccessContext {
        project_id: request
            .project_id
            .clone()
            .or_else(|| agent.project_id.clone()),
        user_id,
        session_id,
    };

    let mut denied_memory_count = 0;
    let mut scored = Vec::new();

    for memory in memories {
        if can_read_memory(agent, memory, &access_context) {
            let scored_memory = score_memory(&request.task, memory);

            if scored_memory.score > 0 {
                scored.push(scored_memory);
            }
        } else {
            denied_memory_count += 1;
        }
    }

    scored.sort_by(sort_scored_memory);

    let (items, estimated_tokens) = pack_context(scored, request.token_budget);
    let allowed_memory_ids = items
        .iter()
        .map(|item| item.memory_id.clone())
        .collect::<Vec<_>>();

    let pack = ContextPack {
        request_id: request_id.clone(),
        agent_id: agent.agent_id.clone(),
        task: request.task.clone(),
        token_budget: request.token_budget,
        estimated_tokens,
        context_pack: items.iter().map(|item| item.content.clone()).collect(),
        items,
    };

    let audit = AuditRecord {
        request_id,
        agent_id: agent.agent_id.clone(),
        org_id: agent.org_id.clone(),
        task: request.task,
        allowed_memory_ids,
        denied_memory_count,
        token_budget: pack.token_budget,
        estimated_tokens,
        created_at_unix_ms,
    };

    ContextBuildOutcome { pack, audit }
```

Die Kommentare und let user_id = None; let session_id = None; sind sehr wichtig. Ein Agent API Key ist kein Beweis fuer einen menschlichen Nutzer oder dessen Browser-Session. Daher darf die Anfrage nicht einfach eine user_id einschmuggeln. Fuer solche privaten Overlays braucht es spaeter einen separaten, signierten Capability Grant.

AccessContext kombiniert die sichere AgentIdentity mit dem erlaubten Projektkontext. Der Ausdruck request.project_id.clone().or_else(|| agent.project_id.clone()) verwendet ein angefragtes Projekt nur, wenn es vorhanden ist, sonst die serverseitige Projektbindung des Agenten. Eine echte Permission-Pruefung

entscheidet danach, ob die Memory passt.

Die for-Schleife ist die Invariante in Codeform: can_read_memory kommt vor score_memory. Nur erlaubte Memory wird zur Bewertung hinzugefuegt. Verbotene Memory erhoeht lediglich einen Zaehler fuer das Audit. Ihr Inhalt wird nicht gerankt oder in den Pack geschoben.

pack_context(scored, request.token_budget) macht aus sortierten Kandidaten einen begrenzten Context Pack. Danach baut der Code ein AuditRecord mit erlaubten IDs, Anzahl abgelehnter Eintraege und Token-Zahlen. Der Audit ist nicht nur Logging - er ist eine fachliche Erklaerung der Entscheidung.

Merksatz

Ein Retrieval-System entscheidet zuerst: Darf ich es sehen? Dann: Ist es relevant? Dann: Passt es ins Budget? Diese Reihenfolge schuetzt Daten und macht Entscheidungen erklaerbar.

22. Embeddings: Text wird zu Suchgeometrie

SOURCE crates/energon-api/src/embedding.rs / 79-112

```
    /// Embed a single query string. Failures are surfaced as errors so callers
    /// can fall back to recency-based retrieval instead of failing requests.
    pub async fn embed(&self, input: &str) -> Result<Vec<f32>, EmbeddingError> {
        let response = self
            .client
            .post(&self.endpoint)
            .bearer_auth(&self.api_key)
            .json(&EmbeddingRequest {
                model: &self.model,
                input: &[input],
            })
            .send()
            .await
            .map_err(|error| EmbeddingError::Request(error.to_string()))?;

        let status = response.status();
        if !status.is_success() {
            return Err(EmbeddingError::Request(format!(
                "OpenAI embeddings returned HTTP {status}"
            )));
        }

        let body = response
            .json::<EmbeddingResponse>()
            .await
            .map_err(|error| EmbeddingError::Malformed(error.to_string()))?;

        body.data
            .into_iter()
            .next()
            .map(|item| item.embedding)
            .filter(|embedding| !embedding.is_empty())
            .ok_or_else(|| EmbeddingError::Malformed("empty embedding data".to_owned()))
    }
```

async fn embed nimmt einen Text und verspricht entweder Result<Vec<f32>, EmbeddingError>. Vec<f32> ist ein dynamisches Array aus 32-Bit-Gleitkommazahlen - genau der Vektor, der spaeter in pgvector gespeichert oder mit anderen Vektoren verglichen werden kann.

client.post(...).bearer_auth(...).json(...).send().await baut einen HTTP POST an den OpenAI Embeddings-Endpunkt. Das API Secret kommt nur aus dem Server-State. Es darf niemals in einen Browser-Bundle oder Agenten-Prompt wandern.

Nach der Antwort prueft is_success() den HTTP-Status und json::<EmbeddingResponse>() prueft die Antwort gegen die erwartete Rust-Struktur. Das sind zwei verschiedene Schutzschichten: Netzwerkstatus und Datenform.

into_iter().next().filter(...).ok_or_else(...) liest den ersten Vektor, verwirft leere Ergebnisse und macht auch das zu einem klaren Fehler. Der Aufrufer kann bei Embedding-Fehlern auf eine einfachere, zum Beispiel zeitbasierte oder Keyword-Suche zurueckfallen, statt den ganzen Context Request kaputtzumachen.

23. Postgres und pgvector: Datenmodell statt lose Textdateien

SOURCE migrations/0002_memory.sql / 1-24, 41-52

```
CREATE TABLE IF NOT EXISTS memory_entries (
    memory_id TEXT PRIMARY KEY,
    org_id TEXT NOT NULL REFERENCES orgs(org_id) ON DELETE CASCADE,
    scope TEXT NOT NULL CHECK (
        scope IN (
            'open',
            'org',
            'project',
            'role',
            'agent_private',
            'user_private',
            'session'
        )
    ),
    content TEXT NOT NULL,
    tags TEXT[] NOT NULL DEFAULT '{}',
    project_id TEXT REFERENCES projects(project_id) ON DELETE CASCADE,
    role_id TEXT REFERENCES roles(role_id) ON DELETE CASCADE,
    owner_agent_id TEXT REFERENCES agents(agent_id) ON DELETE SET NULL,
    user_id TEXT,
    session_id TEXT,
    source TEXT,
    promoted_from TEXT REFERENCES memory_entries(memory_id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()

CREATE TABLE IF NOT EXISTS memory_chunks (
    chunk_id TEXT PRIMARY KEY,
    memory_id TEXT NOT NULL REFERENCES memory_entries(memory_id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(memory_id, chunk_index)
);

CREATE INDEX IF NOT EXISTS memory_chunks_embedding_idx
    ON memory_chunks USING hnsw (embedding vector_cosine_ops);
```

CREATE TABLE memory_entries definiert die Haupttabelle. memory_id TEXT PRIMARY KEY macht jede Memory eindeutig. org_id ... REFERENCES orgs ist ein Fremdschluessel: Eine Memory darf nicht zu einer Organisation zeigen, die es nicht gibt. ON DELETE CASCADE bedeutet, dass beim Loeschen einer Organisation ihre untergeordneten Eintraege mit entfernt werden.

scope TEXT NOT NULL CHECK (...) beschraenkt den Scope auf bekannte Varianten. Das ist eine zweite Verteidigung neben Rust-Enums: Auch ein direkter SQL-Fehler kann nicht einfach scope = 'everyone_forever' speichern.

promoted_from speichert die Herkunft einer freigegebenen Memory. Dadurch kann ein Audit nachvollziehen: Diese Projekt-Memory entstand aus jener urspruenglich privaten Memory.

memory_chunks teilt lange Inhalte in kleinere Teile. Jeder Chunk kann einen embedding vector(1536) tragen. Der hnsw-Index mit vector_cosine_ops beschleunigt die Suche nach semantisch aehnlichen Vektoren. Wichtig: Der Index verbessert Geschwindigkeit, ersetzt aber nie die Berechtigungspruefung.

24. SDK: Ergonomie ohne Identitaetsfaelschung

SOURCE packages/sdk-typescript/src/index.ts / 298-348

```
/**
 * A server-side client for one authenticated agent in one swarm. Agent identity
 * is always derived by the control plane from the API key, never from SDK input.
 */
export class Energon {
  readonly memory: {
    remember: (input: RememberInput, options?: RequestOptions) => Promise<AgentMemory>;
    share: (input: ShareMemoryInput, options?: RequestOptions) => Promise<AgentMemory>;
  };

  readonly context: {
    build: (input: BuildContextInput, options?: RequestOptions) => Promise<ContextPack>;
  };

  readonly audit: {
    context: (requestId: string, options?: RequestOptions) => Promise<ContextAudit>;
    promotion: (memoryId: string, options?: RequestOptions) => Promise<PromotionAudit>;
  };

  readonly swarm: {
    runtime: (options?: RequestOptions) => Promise<SwarmRuntime>;
  };

  readonly claims: {
    assert: (input: AssertClaimInput, options?: RequestOptions) => Promise<ClaimAssertion>;
  };

  readonly skills: {
    /** Returns only skill profiles assigned to the authenticated agent. */
    list: (options?: RequestOptions) => Promise<SkillProfile[]>;
    /** Creates a private profile for the authenticated agent only. */
    create: (input: CreateSkillInput, options?: RequestOptions) => Promise<SkillProfile>;
  };

  readonly operations: {
    /** Agent-safe organization state and the dashboard's live operational metrics. */
    overview: (options?: RequestOptions) => Promise<AgentOperationalOverview>;
  };

  private readonly baseUrl: string;
  private readonly apiKey: string;
  private readonly paymentSignature?: PaymentSignatureProvider;
  private readonly timeoutMs: number;
  private readonly fetchFn: typeof fetch;

  constructor(options: EnergonClientOptions) {
    this.baseUrl = normalizeBaseUrl(options.baseUrl);
    this.apiKey = requiredText(options.apiKey, "apiKey");
    this.paymentSignature = options.paymentSignature;
    this.timeoutMs = normalizeTimeout(options.timeoutMs);
    this.fetchFn = options.fetch ?? globalThis.fetch;
```

Der Kommentar ueber der Klasse beschreibt den Vertrag: Ein Energon-Objekt repraesentiert genau einen authentifizierten Agenten. readonly memory, context, audit und weitere Bereiche geben dem Entwickler eine klare, typisierte API.

Die Methoden nehmen fachliche Eingaben wie RememberInput oder BuildContextInput. Auffaellig fehlt: org_id, role_id oder agent_id. Das ist Absicht. Ein SDK-Nutzer darf die Aufgabe und den Inhalt benennen, aber nicht seine Berechtigungsidentitaet behaupten.

Im Konstruktor normalisiert normalizeBaseUrl die Serveradresse, requiredText prueft, ob ein Key existiert, und normalizeTimeout setzt ein erwartbares Zeitlimit. Das sind kleine Details, die Integrationen robust machen und Fehler frueh sichtbar machen.

TypeScript hilft hier vor allem dem SDK-Nutzer. Die letzte Sicherheitsentscheidung bleibt trotzdem im Rust-Backend, weil jeder Client theoretisch veraendert oder umgangen werden kann.

25. Worker: langsame Nebenarbeit entkoppeln

SOURCE crates/energon-worker/src/main.rs / 63-83

```
    loop {
        let embedded = match &openai_api_key {
            Some(api_key) => {
                process_embedding_batch(&pool, &client, api_key, &model, batch_size).await?
            }
            None => 0,
        };
        let published = match &publisher {
            Some(publisher) => publish_outbox_batch(&pool, publisher, event_batch_size).await?,
            None => 0,
        };

        if run_once {
            tracing::info!(embedded, published, "worker one-shot finished");
            break;
        }

        if embedded == 0 && published == 0 {
            tokio::time::sleep(Duration::from_secs(2)).await;
        }
    }
```

Die loop ist eine dauerhafte Worker-Schleife. Wenn ein OpenAI-Key konfiguriert ist, verarbeitet sie eine kleine Embedding-Batch. Wenn ein NATS Publisher konfiguriert ist, veroeffentlicht sie eine Outbox-Batch. Beide Ergebnisse werden als Zahlen gespeichert: embedded und published.

run_once ist eine kontrollierte Betriebsoption. In Tests, Jobs oder beim manuellen Debugging darf der Worker nach einem Durchlauf sauber enden. In Produktion laeuft er weiter.

Wenn weder Embeddings noch Events Arbeit fanden, wartet tokio::time::sleep zwei Sekunden. Das verhindert eine enge CPU-Schleife, die ununterbrochen dieselbe leere Datenbank abfragt. Weil sleep awaited wird, blockiert er nicht den ganzen Tokio-Thread.

End-to-end: eine echte Context-Anfrage

1. Agent	Ein Agent ruft context.build im SDK auf. Der Key reist im Authorization Header mit.
2. API Edge	Axum Router, Limits und CORS nehmen die Anfrage an. Auth hasht den Key und laedt die AgentIdentity.
3. Daten	Postgres liefert begrenzte Kandidaten fuer Organisation, Projekt, Rolle und Besitzer. Ein Vektorindex kann Relevanz beschleunigen.
4. Core	can_read_memory filtert. Erst danach wird bewertet, sortiert und innerhalb des Token Budgets gepackt.
5. Audit	Erlaubte IDs, abgelehnte Anzahl, Aufgabe und Token-Zahlen werden gespeichert. Der Agent erhaelt nur seinen Pack.

26. Was du als Naechstes lernen solltest

Diese Reihenfolge ist absichtlich praktisch. Nach jedem Schritt solltest du nicht nur lesen, sondern eine kleine Frage am echten Code beantworten.

Woche 1	Rust-Basics: Variablen, Structs, Enums, Match, Option, Result, Vec. Lies energon-core/src/memory.rs und identity.rs.
Woche 2	Ownership und async: Erklaere jedes &, clone() und .await im Context Broker mit eigenen Worten.
Woche 3	HTTP und Axum: Folge einer Route von app.rs bis zum Handler. Teste einen lokalen Request mit absichtlich falschem Key.
Woche 4	SQL und Datenmodell: Lies die Migrations. Zeichne Memory, Agent, Projekt und Audit als Beziehungen auf Papier.
Woche 5	AI Retrieval: Schreibe fuer dich auf, warum Embedding-Aehnlichkeit nie eine Berechtigung ist. Vergleiche Keyword- und Vektorsuche.
Woche 6	Betrieb und Security: Untersuche Environment Variables, Logs, Rate Limits, Outbox und die Fehlerfaelle eines Worker-Restarts.

Abschluss

Du musst nicht jede Zeile sofort verstehen. Das Ziel ist ein belastbares Modell: Energon ist eine Rust-Control-Plane, die Identitaet serverseitig ableitet, Memory nach Berechtigung filtert, AI Retrieval nur fuer erlaubte Kandidaten nutzt und jede Context-Entscheidung pruefbar macht.

> Wenn du bei einer neuen Datei feststeckst, stelle immer dieselben Fragen: Welche Daten kommen hinein? Welcher Typ repraesentiert sie? Welche Schicht besitzt die Entscheidung? Was passiert bei Fehlern? Welche Invariante darf nie gebrochen werden? Damit liest du dich Schritt fuer Schritt zu echtem Senior-Systemverstaendnis vor.

TEIL 4 / RUST UND COMPUTER SCIENCE TIEF

Vom Speicher bis zur async Runtime

Jetzt verbinden wir Sprache, Compiler, CPU, Speicher und Nebenlaeufigkeit. Jede Rust-Regel bekommt einen Grund im echten Computersystem.

27. Vom Quellcode zum Prozess

Cargo liest den Workspace, loest Abhaengigkeiten auf und ruft den Rust-Compiler auf. Der Compiler parst den Text, prueft Typen, Ownership und Borrowing, erzeugt eine Zwischendarstellung und daraus Maschinen-Code fuer die Ziel-CPU. In Produktion laeuft deshalb nicht die Datei main.rs. Das Betriebssystem laedt eine kompilierte Binärdatei in einen neuen Prozess.

Ein Prozess besitzt einen virtuellen Adressraum. Darin liegen Maschinen-Code, statische Daten, Stacks und Heap-Bereiche. Virtuell bedeutet: Die Adressen des Prozesses sind nicht direkt dieselben wie physische RAM-Adressen. Das Betriebssystem und die CPU uebersetzen sie ueber Seitentabellen. Dadurch koennen Prozesse voneinander isoliert werden.

Compiler	Findet viele Fehler vor dem Start und erzeugt Maschinen-Code.
Prozess	Eine laufende Instanz des Programms mit eigenem virtuellen Speicher.
Thread	Ein Ausfuehrungsstrom innerhalb eines Prozesses mit eigenem Stack.
Task	Eine von Tokio geplante async Zustandsmaschine; nicht identisch mit einem Betriebssystemthread.
Socket	Eine Betriebssystemressource fuer Netzwerkkommunikation.

Beim Start von energon-api entstehen Tokio-Runtime-Threads, ein PostgreSQL-Connection-Pool und ein TCP-Listener. Danach wartet der Prozess nicht passiv auf nur einen Client. Er kann viele offene Verbindungen verwalten und Tasks immer dann weiter ausfuehren, wenn deren I/O bereit ist.

28. Stack, Heap und ein Rust String

Der Stack speichert typischerweise lokale Werte und Funktionsrahmen. Er ist sehr schnell, weil Speicher in einer einfachen Reihenfolge reserviert und freigegeben wird. Der Heap speichert dynamisch grosse Werte, deren Groesse oder Lebensdauer nicht schon durch den unmittelbaren Funktionsrahmen bestimmt ist.

Ein Rust String besteht konzeptionell auf dem Stack aus Pointer, Laenge und Kapazitaet. Die UTF-8-Bytes liegen auf dem Heap. Wird der String bewegt, werden Pointer, Laenge und Kapazitaet uebertragen - nicht automatisch alle Textbytes kopiert.

Darum invalidiert let b = a; bei einem String normalerweise den Namen a. Nur b besitzt danach den Heap-Speicher. Rust verhindert so, dass zwei Besitzer denselben Speicher freigeben. clone() erzeugt bewusst eine zweite Kopie und macht die Kosten sichtbar.

Kleine Typen wie u64 implementieren Copy. Bei ihnen ist eine Bitkopie billig und eindeutig. Ein u64 benoetigt keinen Heap und besitzt keine Ressource, die genau einmal freigegeben werden muss.

Cache-Lokalitaet

Die CPU ist viel schneller als der Arbeitsspeicher. Sie besitzt kleine Cache-Ebenen. Zusammenhaengende Daten wie ein Vec koennen cachefreundlicher sein als viele weit verteilte Heap-Objekte. Das ist einer der Gruende, warum Datenlayout bei sehr hoher Last relevant wird - aber erst messen, dann optimieren.

29. Ownership im echten Memory Record

SOURCE crates/energon-core/src/memory.rs / 96-127

```
impl MemoryRecord {
    pub fn from_write(
        memory_id: impl Into<String>,
        agent: &AgentIdentity,
        request: WriteMemoryRequest,
        created_at_unix_ms: u128,
    ) -> Result<Self, EnergonError> {
        request.validate()?;

        let record = Self {
            memory_id: memory_id.into(),
            org_id: agent.org_id.clone(),
            scope: request.scope,
            content: request.content.trim().to_owned(),
            tags: clean_tags(request.tags),
            // The authenticated agent identity owns the routing metadata. An
            // agent must not be able to label a private note as another
            // project or role and later promote it into that scope.
            project_id: agent.project_id.clone(),
            role_id: agent.role_id.clone(),
            owner_agent_id: Some(agent.agent_id.clone()),
            user_id: None,
            session_id: None,
            source: clean_optional(request.source),
            promoted_from: None,
            created_at_unix_ms,
        };

        record.validate_scope_metadata()?;

        Ok(record)
    }
```

agent: &AgentIdentity ist eine unveraenderliche Leihgabe. Die Funktion darf die Identitaet lesen, aber nicht aus dem Aufrufer entfernen. request: WriteMemoryRequest wird dagegen bewegt. Die Funktion darf seine Strings und Vektoren in den neuen Record uebernehmen.

Der Rueckgabetyp Result<Self, EnergonError> sagt: Entweder entsteht ein gueltiger MemoryRecord oder ein expliziter Domain-Fehler. request.validate()? beendet die Funktion sofort, falls die Invariante nicht gilt.

agent.org_id.clone() ist notwendig, weil der Agent nur geliehen wurde. Einen String kann man nicht aus einer geliehenen Struct herausbewegen. Clone erzeugt einen neuen Besitzer. Dagegen kann request.scope direkt uebernommen werden, weil request der Funktion gehoert.

Self { ... } baut einen Wert desselben Typs wie der impl-Block. Der Record wird erst nach einer zweiten Metadatenvalidierung als Ok zurueckgegeben. Das ist eine kleine, aber wichtige Domain-Grenze: Aus der Funktion soll kein halbgueltiger Record entkommen.

30. Referenzen und Lifetimes

Eine Referenz ist kein eigener Besitzer. Sie zeigt temporaer auf einen gueltigen Wert. Der Borrow Checker beweist zur Compile-Zeit, dass eine Referenz nicht laenger lebt als ihr Ziel und dass Schreibzugriffe nicht unkontrolliert mit anderen Zugriffen konkurrieren.

&T	Viele unveraenderliche Leser duerfen gleichzeitig existieren.
&mut T	Genau ein exklusiver Zugriff darf den Wert veraendern.
'a	Ein Lifetime-Parameter beschreibt die Beziehung zwischen Referenzlebenszeiten.
&str	Geliehene Sicht auf gueltigen UTF-8-Text.
&[T]	Geliehene Sicht auf eine zusammenhaengende Sequenz.

Lifetimes sind keine Timer. Sie erzeugen normalerweise keine Runtime-Arbeit. Sie sind Beweise fuer den Compiler. Wenn eine Funktion eine Referenz zurueckgibt, muss klar sein, aus welchem Eingabewert diese Referenz stammt und wie lange sie gueltig bleibt.

Im Context Broker ist memories: &[MemoryRecord] besonders passend. Die Domain-Funktion soll Kandidaten lesen, aber nicht die Datenbankliste besitzen oder veraendern. Eine Slice kann auf einen ganzen Vec oder nur einen Ausschnitt zeigen.

Borrowing und Architektur

Ownership ist nicht nur Speichersicherheit. Eine Funktionssignatur dokumentiert Verantwortung. Besitz bedeutet oft: Diese Schicht darf den Wert weiterreichen oder dauerhaft behalten. Borrowing bedeutet: Diese Schicht betrachtet den Wert nur waehrend der Operation.

31. Enums, Pattern Matching und unmoegliche Zustaende

SOURCE crates/energon-core/src/permissions.rs / 13-35

```rust
pub fn can_read_memory(
    agent: &AgentIdentity,
    memory: &MemoryRecord,
    context: &AccessContext,
) -> bool {
    if memory.org_id != agent.org_id {
        return false;
    }

    match memory.scope {
        MemoryScope::Open | MemoryScope::Org => true,
        MemoryScope::Project => {
            let project_id = context.project_id.as_ref().or(agent.project_id.as_ref());
            memory.project_id.as_ref() == project_id
        }
        MemoryScope::Role => memory.role_id == agent.role_id,
        MemoryScope::AgentPrivate => memory.owner_agent_id.as_ref() == Some(&agent.agent_id),
        MemoryScope::UserPrivate => context.user_id.is_some() && memory.user_id == context.user_id,
        MemoryScope::Session => {
            context.session_id.is_some() && memory.session_id == context.session_id
        }
    }
}
```

Der erste Vergleich ist die Tenant-Grenze. Selbst ein Open-Scope darf im aktuellen Modell nicht organisationsuebergreifend gelesen werden. Danach deckt match jede MemoryScope-Variante ab.

Das Zeichen | verbindet Patterns: Open oder Org fuehren zum selben Ergebnis. Der Project-Arm berechnet zunaechst die wirksame Projekt-ID. as_ref() wandelt Option<String> in Option<&String>, damit kein String bewegt oder kopiert werden muss.

Beim AgentPrivate-Arm wird eine Option-Referenz mit Some(&agent.agent_id) verglichen. Nur der Besitzer erhaelt Zugriff. UserPrivate und Session verlangen zusaetzlich, dass ein vertrauenswuerdiger AccessContext ueberhaupt vorhanden ist.

Enums helfen, unmoegliche oder unerkannte Zustaende zu vermeiden. Wird spaeter ein neuer Scope ergaenzt, meldet der Compiler alle nicht vollstaendigen match-Ausdruecke. Das ist eine Form von architektonischer Wartbarkeit.

32. Traits, Generics und Into

SOURCE crates/energon-core/src/identity.rs / 11-24

```
impl AgentIdentity {
    pub fn new(
        agent_id: impl Into<String>,
        org_id: impl Into<String>,
        role_id: Option<String>,
        project_id: Option<String>,
    ) -> Self {
        Self {
            agent_id: agent_id.into(),
            org_id: org_id.into(),
            role_id,
            project_id,
        }
    }
```

Ein Trait beschreibt Faehigkeiten. Into<String> bedeutet: Dieser Eingabetyp kann in einen String umgewandelt werden. Deshalb akzeptiert der Konstruktor sowohl String als auch viele geliehene Textformen, ohne mehrere Konstruktoren zu benoetigen.

Generics erzeugen typsicheren wiederverwendbaren Code. Bei statischer Dispatch kann der Compiler fuer konkrete Typen spezialisierten Maschinen-Code erzeugen. Das nennt man Monomorphisierung. Der Komfort einer Abstraktion muss daher nicht automatisch einen virtuellen Methodenaufruf zur Laufzeit kosten.

Das Rust-SDK verwendet send<Body: Serialize, Response: DeserializeOwned>. Der Request-Typ muss JSON-serialisierbar sein; der Response-Typ muss aus JSON als besitzender Wert entstehen koennen. Der Compiler prueft den Vertrag an jedem Aufruf.

Trait Objects

Box<dyn Error + Send + Sync> ist dynamischer Dispatch. Der konkrete Fehlertyp kann variieren, aber alle Werte erfuellen den Error-Vertrag und duerfen sicher ueber Thread-Grenzen verwendet werden. Das ist an Integrationsgrenzen praktisch, waehrend Domain-Enums praeziser bleiben sollten.

33. Async, Futures und Tokio

Eine async Funktion wird beim Aufruf nicht sofort vollstaendig ausgefuehrt. Sie erzeugt eine Future. Diese Future ist eine Zustandsmaschine, die gespeichert hat, an welcher await-Stelle sie pausiert und welche lokalen Werte sie spaeter noch benoetigt.

Ein Executor pollt Futures. Ist eine Netzwerkoperation noch nicht bereit, liefert die Future Pending. Das Betriebssystem benachrichtigt Tokio spaeter ueber neue I/O-Bereitschaft. Dann wird die Future erneut gepollt. So verwalten wenige Runtime-Threads viele offene Requests.

Concurrency	Viele Aufgaben befinden sich gleichzeitig im Fortschritt.
Parallelism	Mehrere CPU-Kerne fuehren zur selben Zeit Instruktionen aus.
I/O-bound	Die meiste Zeit wartet die Aufgabe auf Netzwerk oder Datenbank.
CPU-bound	Die meiste Zeit rechnet die CPU; hier helfen Threads oder spezialisierte Worker.
Backpressure	Produzenten muessen gebremst werden, wenn Verbraucher nicht nachkommen.

Blockierende Arbeit darf nicht lange auf Tokio-Worker-Threads laufen. Ein synchroner SDK-Client oder teure CPU-Berechnung kann den Thread festhalten und viele andere Tasks verzoegern. Dafuer gibt es async Clients, spawn_blocking oder separate Worker-Pools.

34. Arc, RwLock und Atomics

SOURCE crates/energon-api/src/state.rs / 19-68

```
#[derive(Clone)]
pub struct AppState {
    pub storage: StorageBackend,
    pub auth: AuthConfig,
    pub x402: X402Config,
    pub billing: Option<BaseCheckoutConfig>,
    pub jwt: Option<JwtVerifier>,
    pub embedding: Option<EmbeddingClient>,
    pub rate_limiter: RateLimiter,
    pub retrieval_candidate_limit: i64,
    next_memory: Arc<AtomicU64>,
    next_promotion: Arc<AtomicU64>,
    next_request: Arc<AtomicU64>,
    next_receipt: Arc<AtomicU64>,
    next_usage_event: Arc<AtomicU64>,
    next_checkout: Arc<AtomicU64>,
    next_claim: Arc<AtomicU64>,
    next_conflict: Arc<AtomicU64>,
    next_skill: Arc<AtomicU64>,
}

#[derive(Clone)]
pub struct AuthConfig {
    pub dev_identity_headers: bool,
    pub admin_token: Option<String>,
    pub api_key_pepper: Option<String>,
}

#[derive(Clone)]
pub enum StorageBackend {
    Memory(InMemoryStorage),
    Postgres(PgPool),
}

/// In-memory usage counters keyed by `(org_id, route)`. Used only when the
/// API runs without Postgres; nothing is persisted (documented behavior).
#[derive(Debug, Clone, Copy, Default)]
pub struct UsageCounter {
    pub calls: u64,
    pub paid_calls: u64,
    pub amount_usdc_micro: u64,
}

#[derive(Clone)]
pub struct InMemoryStorage {
    pub memories: Arc<RwLock<Vec<MemoryRecord>>>,
    pub audits: Arc<RwLock<HashMap<String, AuditRecord>>>,
    pub promotion_audits: Arc<RwLock<HashMap<String, PromotionAuditRecord>>>,
    pub usage: Arc<RwLock<HashMap<(String, String), UsageCounter>>>,
}
```

Arc ist atomar referenzgezaehlter gemeinsamer Besitz. Ein Clone des AppState kopiert nicht alle Memories. Er erhoeht bei Arc-Feldern nur einen sicheren Zaehler. Der Wert wird freigegeben, wenn der letzte Besitzer verschwindet.

RwLock erlaubt mehrere Leser oder genau einen Schreiber. Die In-Memory-Variante eignet sich fuer lokale Entwicklung, aber nicht als globaler Produktionsspeicher: Replikate teilen sie nicht, Neustarts loeschen sie und ein grosser Vec erzeugt lineare Arbeit.

AtomicU64 bietet atomare Zaehleroperationen. Ordering::Relaxed reicht fuer eine lokale Nummer, weil keine weiteren Speicheroperationen daran geordnet werden muessen. Die Kombination aus Unix-Millisekunden und Prozesszaehler kollidiert jedoch zwischen Replikaten. Fuer horizontale Skalierung sind UUIDv7 oder ULID besser.

TEIL 5 / POSTGRESQL UND VECTOR RETRIEVAL

Daten dauerhaft, konsistent und auffindbar machen

Diese Schicht entscheidet, welche Wahrheiten das System bewahren kann, wie konkurrierende Writes zusammenleben und wie semantische Suche messbar wird.

35. Das relationale Modell von Energon

Eine relationale Datenbank speichert nicht nur lose Dokumente. Tabellen beschreiben Entitaeten, Foreign Keys ihre Beziehungen und Constraints die erlaubten Zustaende. Energon trennt Identitaet, Memory, Context Audits, Events, Claims, Billing und Skill-Profile in eigene fachliche Bereiche.

Identity	orgs, projects, roles, agents und agent_api_keys.
Memory	memory_entries, memory_chunks und memory_promotions.
Context	context_requests und context_request_items.
Events	event_outbox als dauerhafte Bruecke zum Message Bus.
Claims	memory_claims, claim_conflicts, evidence und Audit Chain.

Eine Primary Key Constraint garantiert Eindeutigkeit und Nicht-Null. Ein Foreign Key verhindert Referenzen auf nicht vorhandene Records. Eine CHECK-Constraint begrenzt beispielsweise Scope oder Confidence. Damit verteidigt PostgreSQL Invarianten auch dann, wenn Anwendungscode einen Fehler macht.

ON DELETE CASCADE ist bequem, aber bei riesigen Tenants potenziell teuer. In einer Plattform mit Milliarden Records wuerde eine Tenant-Loeschung als kontrollierter Workflow laufen: Tenant deaktivieren, neue Writes stoppen, Partitionen oder Batches loeschen, Auditnachweise erstellen und das Ergebnis verifizieren.

36. Indizes wirklich verstehen

Ein Index ist eine zusaetzliche Datenstruktur. Er macht bestimmte Reads schneller, kostet aber Speicher, Cache und Schreibarbeit. Ein Index ist deshalb keine kostenlose Beschleunigung fuer alles.

B-Tree	Gleichheit, Sortierung und Wertebereiche. Beispiel: org_id plus created_at.
Partial Index	Enthaelt nur relevante Zeilen, etwa nicht widerrufene API Keys.
GIN Trigram	Beschleunigt Textaehnlichkeit und bestimmte LIKE-Suchen.
HNSW	Approximate Nearest Neighbor Suche fuer Embedding-Vektoren.
Composite Index	Mehrere Spalten; Reihenfolge und Query-Form sind entscheidend.

Der Index (org_id, created_at DESC) passt zu einer Query mit WHERE org_id und ORDER BY created_at DESC. Er ist nicht automatisch optimal fuer eine Query, die nur nach created_at sucht. Der linke Praefix eines zusammengesetzten B-Tree-Index bestimmt viele seiner moeglichen Zugriffswege.

Mit EXPLAIN ANALYZE prueft man den echten Plan: Index Scan oder Sequential Scan, geschaetzte und tatsaechliche Zeilenzahlen, Sortierung, Buffer-Zugriffe und Laufzeit. SQL-Text allein beweist nicht, welche physische Reihenfolge PostgreSQL ausfuehrt.

37. Connection Pools und globale Mathematik

SOURCE crates/energon-db/src/pool.rs / 1-8

```rust
use sqlx::{PgPool, postgres::PgPoolOptions};

pub async fn connect(database_url: &str) -> Result<PgPool, sqlx::Error> {
    PgPoolOptions::new()
        .max_connections(10)
        .connect(database_url)
        .await
}
```

Eine PostgreSQL-Verbindung besitzt serverseitigen Zustand und kostet Speicher. Ein Pool verwendet wenige Verbindungen fuer viele Requests wieder. Wenn alle Verbindungen belegt sind, wartet ein Request auf den Pool - diese Wartezeit muss gemessen werden.

Zehn Verbindungen wirken klein. Hundert API-Replikate erzeugen aber bis zu tausend moegliche Verbindungen. Systemdesign rechnet immer global, nicht nur pro Pod. PgBouncer kann viele kurzlebige Client-Sessions auf eine kontrollierte Anzahl echter PostgreSQL-Verbindungen multiplexen.

Zu viele Verbindungen machen PostgreSQL nicht schneller. Sie erhoehen Kontextwechsel, Speicherverbrauch und konkurrierende Arbeit. Der richtige Pool folgt gemessenem Query-Durchsatz, CPU, I/O und Latenzzielen.

Read Replicas

Read Replicas entlasten den Primary, sind aber normalerweise leicht verzoegert. Ein unmittelbar nach einem Write ausgefuehrter Read kann den neuen Wert dort noch nicht sehen. Auth-Revocation und sicherheitskritische Entscheidungen brauchen daher besondere Konsistenzregeln.

38. Transaktionen und Isolation

SOURCE crates/energon-db/src/memory.rs / 8-17

```
pub async fn insert_memory(pool: &PgPool, record: &MemoryRecord) -> Result<(), DbError> {
    identity::ensure_memory_record_refs(pool, record).await?;

    let mut tx = pool.begin().await?;
    insert_memory_in_tx(&mut tx, record).await?;
    event_outbox::enqueue_in_tx(&mut tx, &ControlPlaneEvent::memory_written(record)).await?;
    tx.commit().await?;

    Ok(())
}
```

Memory, Chunk und Outbox-Event muessen gemeinsam sichtbar werden. Die Transaktion garantiert Atomicity: Entweder Commit macht alle Aenderungen sichtbar oder ein Fehler rollt alles zurueck.

PostgreSQL verwendet standardmaessig READ COMMITTED. Jede Anweisung sieht einen zum Start dieser Anweisung bestaetigten Zustand. Zwei Anweisungen derselben Transaktion koennen daher unterschiedliche neu bestaetigte Daten sehen. Fuer einfache Inserts ist das meist ausreichend; Read-Modify-Write-Entscheidungen benoetigen Locks oder staerkere Isolation.

Lost Update	Zwei Writer lesen denselben Wert und ueberschreiben einander.
Write Skew	Zwei Transaktionen halten einzeln Regeln ein, verletzen sie aber gemeinsam.
Deadlock	Transaktionen warten zyklisch aufeinander; PostgreSQL bricht eine ab.
Lock Wait	Eine Transaktion wartet auf eine gehaltene Sperre; lange Transaktionen verschaerfen das.

Claims verwenden eine transaktionsgebundene Advisory Lock pro Organisation, Subject und Predicate. Damit entscheidet immer nur eine Transaktion zur selben semantischen Tatsache. Das ist ein Single-Writer-Prinzip auf feiner fachlicher Granularitaet.

39. Embeddings als Geometrie

Ein Embedding ist ein Vektor mit vielen Gleitkommazahlen. Das aktuelle Schema speichert 1.536 Dimensionen. Die einzelne Dimension hat fuer Menschen selten eine stabile isolierte Bedeutung; relevant ist die Lage des gesamten Vektors im Raum.

Cosine Similarity vergleicht die Richtung zweier Vektoren. Das Skalarprodukt wird durch die beiden Vektorlaengen geteilt. Bei aehnlicher Richtung ist die Similarity hoch. pgvector stellt mit dem verwendeten Operator eine Cosine Distance bereit, bei der kleinere Werte bessere Treffer bedeuten.

Semantische Aehnlichkeit ist keine Wahrheit und keine Berechtigung. Ein Text kann thematisch aehnlich, aber falsch, veraltet, untrusted oder privat sein. Retrieval liefert Kandidaten; Domain-Regeln entscheiden, was verwendet werden darf.

Embedding	Numerische Repraesentation eines Textes.
Query Vector	Embedding der aktuellen Aufgabe oder Suchfrage.
Distance	Mathematische Entfernung zwischen Query und gespeichertem Vektor.
Top K	Anzahl der besten Kandidaten, die eine Suche liefert.
Recall@K	Wie viele wirklich relevante Ergebnisse in den Top K gefunden wurden.

40. HNSW und Approximate Search

Eine exakte Vector Search vergleicht den Query-Vektor mit jedem gespeicherten Vektor. Bei N Vektoren und D Dimensionen ist die rohe Arbeit grob proportional zu N mal D. Das wird bei Milliarden Vektoren teuer.

HNSW baut einen mehrstufigen Graphen. Obere Ebenen erlauben grosse Spruenge, untere Ebenen verfeinern die Nachbarschaft. Die Suche navigiert von groben zu nahen Kandidaten, statt jeden Punkt vollstaendig zu pruefen.

Die Suche ist approximate: Sie ist schnell und hat oft hohen Recall, garantiert aber nicht immer das exakte mathematische Optimum. Parameter beeinflussen den Trade-off zwischen Latenz, Recall, Indexgroesse und Write-Kosten.

Bei Multi-Tenant-Filtern entsteht ein schwieriger Punkt: Ein globaler ANN-Index kann nahe Vektoren anderer Tenants finden, die danach weggefiltert werden. Dann bleiben eventuell zu wenige erlaubte Treffer. Partitionierung, iterative Scans, Tenant-Zellen oder spezialisierte Filtered-ANN-Systeme muessen mit realistischen Daten evaluiert werden.

41. Die aktuelle semantische SQL-Pipeline

SOURCE crates/energon-db/src/memory.rs / 364-419

```
        WITH allowed AS (
            SELECT memory_id, created_at
            FROM memory_entries
            WHERE org_id = $1
              AND (
                scope IN ('open', 'org')
                OR (scope = 'project' AND project_id = $2)
                OR (scope = 'role' AND role_id = $3)
                OR (scope = 'agent_private' AND owner_agent_id = $4)
                OR (scope = 'user_private' AND user_id = $5)
                OR (scope = 'session' AND session_id = $6)
              )
        ),
        semantic AS (
            SELECT allowed.memory_id
            FROM allowed
            INNER JOIN memory_chunks ON memory_chunks.memory_id = allowed.memory_id
            WHERE memory_chunks.embedding IS NOT NULL
            GROUP BY allowed.memory_id
            ORDER BY min(memory_chunks.embedding <=> $7::vector) ASC
            LIMIT $8
        ),
        unembedded AS (
            SELECT allowed.memory_id
            FROM allowed
            WHERE NOT EXISTS (
                SELECT 1
                FROM memory_chunks
                WHERE memory_chunks.memory_id = allowed.memory_id
                  AND memory_chunks.embedding IS NOT NULL
            )
            ORDER BY allowed.created_at DESC
            LIMIT $8
        )
        SELECT
            memory_id,
            org_id,
            scope,
            content,
            tags,
            project_id,
            role_id,
            owner_agent_id,
            user_id,
            session_id,
            source,
            promoted_from,
            floor(extract(epoch from created_at) * 1000)::bigint AS created_at_unix_ms
        FROM memory_entries
        WHERE memory_id IN (
            SELECT memory_id FROM semantic
            UNION
            SELECT memory_id FROM unembedded
        )
        ORDER BY created_at DESC
        "#,
```

Die CTE allowed begrenzt zuerst Organisation und Scopes. semantic verbindet nur diese IDs mit eingebetteten Chunks und sortiert nach der kleinsten Distanz eines Chunks. unembedded nimmt frische Records ohne fertiges Embedding nach Zeit auf, damit Worker-Lag neue Memory nicht unsichtbar macht.

Das ist ein guter Degradation-Pfad: Embeddings sind eventual consistent, die fachliche Memory aber sofort vorhanden. Der Request darf auch dann funktionieren, wenn der Embedding-Provider oder Worker hinterherhinkt.

Die aktuelle Query gibt den Semantic Score jedoch nicht bis zum Core weiter. Die finale Auswahl wird nach created_at sortiert, und der Core filtert danach noch auf Keyword-Score groesser null. Damit ist pgvector aktuell Candidate Generator, nicht finaler Ranker.

Bessere Hybrid-Pipeline

Permission Filter -> Semantic Top K -> BM25 oder Full-Text Score -> Recency und Source Quality -> Reranking -> Diversifikation -> Token Packing. Jede Stufe sollte ihre Scores und Gruende fuer Evaluation und Audit transportieren.

42. Chunking, Modellversionen und Re-Embedding

Der aktuelle Write-Pfad erzeugt einen Chunk mit Index null. Lange Dokumente benoetigen eine echte Chunking-Strategie. Zu grosse Chunks verwischen mehrere Themen; zu kleine verlieren Kontext und erhoehen Indexmenge sowie Kosten.

Chunk Size	Nach Tokens statt Zeichen messen; passend zum Modell und Dokumenttyp.
Overlap	Ein kleiner Ueberlapp verhindert harte Bedeutungsbrueche, erzeugt aber Duplikate.
Metadata	Titel, Abschnitt, Quelle, Tenant und Version am Chunk bewahren.
Deduplication	Nahezu gleiche Chunks nicht mehrfach in den Pack aufnehmen.
Versioning	Modell, Dimensionen, Chunker-Version und embedded_at speichern.

Bei einem Modellwechsel sollte man nicht die alte Spalte blind ueberschreiben. Ein sicherer Ablauf schreibt neue Embeddings parallel, baut einen neuen Index, vergleicht Recall und Latenz, schaltet Reads kontrolliert um und entfernt die alte Version spaeter.

Mehrere Embedding-Worker benoetigen Leasing wie die Outbox. Sonst koennen sie dieselben NULL-Chunks gleichzeitig waehlen und doppelte Provider-Kosten verursachen.

TEIL 6 / VERTEILTE SYSTEME

Fehler sind ein normaler Betriebszustand

Netzwerke koennen langsam sein, Antworten koennen verloren gehen und Prozesse koennen zwischen zwei Zeilen abstuerzen. Das Design muss diese Realitaet ausdruecken.

43. Timeouts, Retries und Idempotenz

Jeder Netzwerkaufruf braucht ein endliches Timeout. Ohne Timeout kann ein Request Ressourcen unbestimmt lange halten. Ein Timeout beweist aber nicht, dass die Gegenseite nichts getan hat. Vielleicht wurde der Write committed und nur die Antwort ging verloren.

Ein automatischer Retry einer Mutation kann deshalb Duplikate erzeugen. Die Loesung ist ein Idempotency Key: Der Client vergibt fuer eine fachliche Operation eine stabile ID, PostgreSQL erzwingt ihre Eindeutigkeit und der Server liefert bei Wiederholung das bereits gespeicherte Ergebnis.

Safe Retry	GET oder fachlich idempotente Operation mit begrenztem Backoff.
Unsafe Retry	Neuen Record erzeugen, ohne Idempotency Key.
Backoff	Wartezeit waechst nach Fehlern, damit das Zielsystem sich erholen kann.
Jitter	Zufaellige Streuung verhindert, dass alle Clients gleichzeitig erneut senden.
Retry Budget	Begrenzt zusaetzliche Last; ein Ausfall darf keinen Retry-Sturm erzeugen.

Timeouts muessen hierarchisch sein. Der aeussere API-Request braucht mehr Budget als einzelne DB- oder Embedding-Aufrufe, aber weniger als der Client. Sonst arbeitet ein Server weiter, obwohl der Client bereits aufgegeben hat.

44. Token Bucket Rate Limiting

SOURCE crates/energon-api/src/middleware/rate_limit.rs / 56-77

```
    /// Take one token for `key` at time `now`. Returns `false` when the
    /// bucket is empty (the caller should answer 429).
    pub fn try_acquire_at(&self, key: &str, now: Instant) -> bool {
        self.maybe_cleanup(now);

        let mut bucket = self.inner.buckets.entry(key.to_owned()).or_insert(Bucket {
            tokens: self.inner.burst,
            last_refill: now,
        });

        let elapsed = now
            .saturating_duration_since(bucket.last_refill)
            .as_secs_f64();
        bucket.tokens = (bucket.tokens + elapsed * self.inner.rps).min(self.inner.burst);
        bucket.last_refill = now;

        if bucket.tokens >= 1.0 {
            bucket.tokens -= 1.0;
            true
        } else {
            false
        }
```

Ein Bucket besitzt Tokens. Zeit fuellt sie mit rps wieder auf, burst begrenzt die maximale Anzahl. Jede Anfrage verbraucht einen Token. Dadurch sind kurze Spitzen erlaubt, dauerhafte Last bleibt begrenzt.

elapsed * rps berechnet neu entstandene Tokens. min(burst) verhindert unbegrenztes Ansparen. Wenn mindestens ein Token vorhanden ist, wird er subtrahiert und der Request darf weiterlaufen; sonst antwortet die Middleware mit 429.

Der aktuelle Limiter lebt pro Prozess. Mit zehn API-Pods erhaelt derselbe Key effektiv etwa das Zehnfache. Fuer horizontale Skalierung gehoert die globale Grenze an ein Gateway oder in einen verteilten, atomaren Store. Lokale Limits bleiben trotzdem als zweite Schutzschicht sinnvoll.

X-Forwarded-For darf nur vertraut werden, wenn ein kontrollierter Proxy externe Werte entfernt und selbst neu setzt. Sonst kann ein Client seine IP-Identitaet faelschen.

45. Das Outbox Pattern

Ein direkter Dual Write an PostgreSQL und NATS besitzt keinen gemeinsamen atomaren Commit. Nach dem Datenbank-Commit kann der Prozess vor dem Publish abstuerzen. Oder der Publish gelingt und der DB-Write nicht. Die Outbox speichert das Event deshalb in derselben Datenbanktransaktion wie die fachliche Mutation.

SOURCE crates/energon-db/src/event_outbox.rs / 59-106

```rust
/// Lease ready events with `SKIP LOCKED` so multiple publisher workers can run
/// safely without handing the same event to NATS concurrently.
pub async fn claim_batch(
    pool: &PgPool,
    worker_id: &str,
    limit: i64,
    lease_seconds: i64,
) -> Result<Vec<PendingOutboxEvent>, DbError> {
    let rows = sqlx::query(
        r#"
        WITH candidates AS (
            SELECT event_id
            FROM event_outbox
            WHERE published_at IS NULL
              AND available_at <= now()
              AND (lease_expires_at IS NULL OR lease_expires_at <= now())
            ORDER BY created_at ASC
            LIMIT $1
            FOR UPDATE SKIP LOCKED
        )
        UPDATE event_outbox AS outbox
        SET attempts = attempts + 1,
            lease_owner = $2,
            lease_expires_at = now() + make_interval(secs => $3::double precision),
            last_error = NULL
        FROM candidates
        WHERE outbox.event_id = candidates.event_id
        RETURNING outbox.event_id, outbox.subject, outbox.payload, outbox.attempts
        "#,
    )
    .bind(limit.clamp(1, 500))
    .bind(worker_id)
    .bind(lease_seconds.clamp(5, 3_600))
    .fetch_all(pool)
    .await?;

    rows.into_iter()
        .map(|row| {
            Ok(PendingOutboxEvent {
                event_id: row.try_get("event_id")?,
                subject: row.try_get("subject")?,
                payload: row.try_get("payload")?,
                attempts: row.try_get("attempts")?,
            })
        })
        .collect::<Result<Vec<_>, sqlx::Error>>()
        .map_err(Into::into)
}
```

FOR UPDATE SKIP LOCKED laesst mehrere Worker unterschiedliche Rows leasen, ohne gegenseitig zu warten. lease_owner und lease_expires_at bilden zeitlich begrenzten Besitz. Stirbt ein Worker, wird das Event nach Ablauf wieder verfuegbar.

Die Query erhoeht attempts beim Leasing. Bei einem Fehler setzt release_after_failure eine spaetere available_at-Zeit und speichert eine gekuerzte Fehlermeldung. Das ist kontrollierte Wiederholung statt enger Fehler-Schleife.

46. At-least-once und Consumer-Idempotenz

Ein Worker kann erfolgreich an JetStream senden und vor mark_published abstuerzen. Danach wird dasselbe Event erneut gesendet. Das System bietet deshalb at-least-once Delivery, nicht magisches Exactly-once.

Nats-Msg-Id hilft innerhalb des Duplicate Windows. Consumer muessen trotzdem dauerhaft idempotent sein. Eine typische Technik ist eine processed_event_ids-Tabelle, die Event-ID und fachliche Aenderung in derselben Consumer-Transaktion speichert.

At-most-once	Keine Duplikate, aber Events koennen verloren gehen.
At-least-once	Events gehen nicht still verloren, koennen aber mehrfach ankommen.
Effectively once	At-least-once Transport plus Idempotenz und Deduplication.
Ordering	Reihenfolge gilt meist nur innerhalb einer definierten Partition oder eines Subjects.

Ein Event-Schema braucht Versionierung und additive Evolution. Alte Consumer koennen neue optionale Felder ignorieren. Entfernte oder umgedeutete Felder benoetigen neue Versionen und Migrationsfenster.

47. Backpressure und Queue Health

Wenn Produzenten schneller schreiben als Worker verarbeiten, waechst die Queue. Das ist anfangs eine Pufferfunktion, spaeter aber ein Kapazitaetsproblem. Entscheidend ist nicht nur die Anzahl pending Events, sondern das Alter des aeltesten Events.

Input Rate	Neue Arbeit pro Sekunde.
Service Rate	Erfolgreich verarbeitete Arbeit pro Sekunde.
Queue Age	Wie lange die aelteste wartende Arbeit bereits liegt.
Retry Rate	Anteil fehlgeschlagener und erneut geplanter Jobs.
Poison Item	Arbeit, die deterministisch immer wieder fehlschlaegt.

Poison Items brauchen Quarantaene oder einen Dead-Letter-Pfad. Unendliche Retries koennen sonst die Queue belasten und echte Arbeit verdraengen. Operatoren brauchen Einsicht, sichere Wiederholung und eine dokumentierte Aufloesung.

Backpressure kann Batches verkleinern, Writes begrenzen, Prioritaeten verwenden oder Clients mit Retry-After abbremsen. Unbegrenzte interne Queues verschieben einen Ausfall nur bis zum Speicher- oder Kostenkollaps.

48. Claims, Konflikte und Single Writer

SOURCE crates/energon-core/src/claims.rs / 14-41

```rust
/// A semantic claim is only promoted automatically when its server-derived
/// authority and agent-supplied confidence exceed the incumbent by 10%.
/// Narrow margins stay contested for a validator or operator to resolve.
pub fn compare_claims(
    same_value: bool,
    incumbent_score: Option<i64>,
    challenger_score: i64,
) -> ClaimResolution {
    if same_value {
        return ClaimResolution::SameValue;
    }

    let Some(incumbent_score) = incumbent_score else {
        return ClaimResolution::NewClaimWins;
    };

    if challenger_score.saturating_mul(10_000) >= incumbent_score.saturating_mul(11_000) {
        ClaimResolution::NewClaimWins
    } else {
        ClaimResolution::Contested
    }
}

pub fn decision_for_resolution(resolution: ClaimResolution) -> ClaimDecision {
    match resolution {
        ClaimResolution::SameValue | ClaimResolution::NewClaimWins => ClaimDecision::Accepted,
        ClaimResolution::Contested => ClaimDecision::Contested,
    }
```

Freier Memory-Text bewahrt Kontext, ist aber nicht automatisch eine globale Tatsache. Structured Claims besitzen Subject, Predicate, JSON-Wert, Evidence, Confidence und serverseitige Authority.

Der Challenger ersetzt einen widerspruechlichen Incumbent nur mit mindestens zehn Prozent Vorsprung. Die Hysterese verhindert Flattern bei kleinen Score-Unterschieden. Nahe Konflikte bleiben contested und werden von einem berechtigten Operator aufgeloest.

Die Berechnung verwendet Integer-Basis-Points und saturating_mul. Saturating Arithmetic laeuft bei Ueberlauf nicht um, sondern bleibt am maximalen Wert. So kann ein extremer Wert die Vergleichslogik nicht durch Wraparound umkehren.

Eine Advisory Lock pro org, subject und predicate serialisiert konkurrierende Entscheidungen. Das ist bewusst kein Multi-Region-CRDT. Es ist eine klare transaktionale Policy mit einem Schreiber pro Konfliktschluessel.

49. Audit Chain und ihre Grenzen

Eine Hash Chain verbindet jeden Audit-Event mit dem vorherigen Hash. Wird ein historischer Payload veraendert, stimmen nachfolgende Hashes nicht mehr. Das macht Manipulation erkennbar und erlaubt eine lineare Integritaetspruefung.

Eine Hash Chain ist nicht automatisch unveraenderbar. Wer volle Schreibrechte auf die gesamte Datenbank besitzt, koennte die Kette neu berechnen. Staerkere Garantien entstehen durch externe Verankerung: regelmaessig einen Root Hash in einem getrennten System, WORM Storage oder einer unabhaengigen Signaturinstanz speichern.

Audit und normales Logging haben andere Zwecke. Logs helfen beim technischen Debugging und duerfen rotiert werden. Audit dokumentiert fachliche Entscheidungen, Akteure, Gruende und verwendete Daten. Retention, Datenschutz und Zugriff muessen deshalb separat entworfen werden.

TEIL 7 / SKALIERUNG

Von hunderttausend bis hundert Millionen Agents

Agentenzahl allein ist keine Lastangabe. Wir rechnen mit aktiven Agents, Operationen, Peak-Faktoren, Datenwachstum, Latenz und Fehlerzielen.

50. Das Lastmodell

Fuer ein vergleichbares Gedankenmodell nehmen wir an: Jeder aktive Agent baut einmal pro Minute Kontext und schreibt einmal pro zehn Minuten Memory. Das ist keine Prognose, sondern eine Rechengrundlage.

100.000 aktiv	Etwa 1.667 Context RPS und 167 Memory Writes pro Sekunde.
1 Million aktiv	Etwa 16.667 Context RPS und 1.667 Writes pro Sekunde.
10 Millionen aktiv	Etwa 166.667 Context RPS und 16.667 Writes pro Sekunde.
100 Millionen aktiv	Etwa 1.666.667 Context RPS und 166.667 Writes pro Sekunde.

Durchschnitt ist nicht Peak. Synchron gestartete Agenten koennen zur vollen Minute einen Burst erzeugen. Dazu kommen Retries, Worker-Queries, Embedding-Provider-Traffic, Audit-Writes und Dashboard-Reads.

Little's Law verbindet Durchsatz, Latenz und Gleichzeitigkeit: concurrency ist ungefaehr RPS mal durchschnittliche Dauer. Bei 10.000 RPS und 200 Millisekunden sind durchschnittlich etwa 2.000 Requests gleichzeitig im System.

51. Datenwachstum und Kosten

Nehmen wir zwanzig Memories pro Agent und Tag, zwei Kilobyte Text und einen 1.536-dimensionalen Float32-Vektor. Ein Rohvektor benoetigt ungefaehr sechs Kilobyte. Metadaten, Row-Overhead, HNSW, WAL, Replikation, Audit und Backups kommen hinzu.

100.000 Agents	2 Millionen Memories pro Tag; grob 16 GB Text plus Rohvektoren.
1 Million	20 Millionen pro Tag; grob 160 GB vor Index- und Replikationsfaktoren.
10 Millionen	200 Millionen pro Tag; grob 1,6 TB vor Overhead.
100 Millionen	2 Milliarden pro Tag; grob 16 TB vor Overhead.

Bei Agent-Systemen wird Daten- und Eventwachstum oft frueher zum Problem als reine HTTP-QPS. Retention, Kompression, Deduplizierung, Cold Storage und Tenant-Quoten gehoeren deshalb zum Produktdesign, nicht nur zum spaeteren Betrieb.

52. Architektur fuer 100.000 aktive Agents

Eine gut gebaute regionale Plattform kann hier mit wenigen stateless API-Replikaten, PgBouncer, einem starken PostgreSQL-Cluster, pgvector, separaten Workern und JetStream arbeiten. Entscheidend sind Grenzen und Messbarkeit, nicht ein frueher Microservice-Zoo.

API	Horizontal repliziert, global eindeutige IDs, Graceful Shutdown.
Rate Limit	Am Gateway verteilt, lokal als zweite Schutzschicht.
Postgres	PgBouncer, passende Indizes, Query Budgets und Backups.
Retrieval	Begrenzte Kandidaten, Hybrid Score und echte Recall-Evaluation.
Worker	Leases, Backoff, Jitter, Queue-Age-Alerts und Quarantaene.

Die wichtigste Arbeit ist hier meist Korrektheit: UUIDv7 oder ULID, Idempotency Keys, Embedding-Leasing, Batch-Audits, exakter Tokenizer, Tracing und Lasttests. Eine einzelne Region reduziert verteilte Konsistenzkomplexitaet.

53. Architektur fuer eine Million aktive Agents

Bei etwa 17.000 Context RPS und starkem Datenwachstum wird eine einzige globale Datenbank zu einem Failure- und Skalierungszentrum. Jetzt wird Cell Architecture interessant.

Eine Cell enthaelt API Compute, Datenbank, Retrieval, Worker und Queue-Kapazitaet fuer eine begrenzte Tenant-Gruppe. Ein kleines globales Directory ordnet org_id einer Home Cell zu. Der normale Request bleibt danach innerhalb dieser Cell.

Failure Isolation	Ein Cell-Ausfall betrifft nur einen begrenzten Tenant-Anteil.
Capacity	Neue Cells erweitern die Plattform in bekannten Einheiten.
Placement	Tenants werden nach Region, Groesse und Compliance zugeordnet.
Hot Tenant	Sehr grosse Kunden koennen eine dedizierte Cell oder Shards erhalten.
Directory Cache	Routing darf nicht fuer jeden Request eine globale DB benoetigen.

Rebalancing ist ein Produkt fuer sich: Daten kopieren, Dual Reads oder kontrollierten Write Freeze verwenden, Checksummen pruefen, Routing atomar umstellen und die alte Kopie spaeter entfernen.

54. Architektur fuer zehn Millionen aktive Agents

Jetzt arbeitet die Plattform in vielen regionalen Cells. Compute und Storage muessen getrennt geplant werden. Nicht jede Cell besitzt dasselbe Lastprofil: einige sind read-heavy, andere erzeugen riesige Memory-Mengen oder Embedding-Bursts.

Vector Layer	Tenant- oder Cell-Shards, spezialisierte ANN-Systeme wenn pgvector-Messungen es verlangen.
Cold Data	Alte Inhalte in Object Storage, nur relevante Metadaten und warme Vektoren online.
Events	Regionale Busse, klare Partition Keys und asynchrone globale Analytics.
Operations	Automatisiertes Provisioning, Rebalancing, Restore und Chaos Tests.
Governance	Regionale Datenhoheit, Retention, Loeschung und Audit Policies.

Globale synchrone Abhaengigkeiten werden vermieden. Eine europaeische Cell soll Context bauen koennen, auch wenn ein globales Analytics-System oder eine amerikanische Region ausgefallen ist.

55. Architektur fuer hundert Millionen aktive Agents

Bei mehr als 1,6 Millionen Context Requests pro Sekunde und Milliarden neuen Memories pro Tag ist Energon eine globale Plattform. Die zentrale Skalierungseinheit lautet Tenant - Region - Cell - Shard.

Anycast oder globales Edge Routing bringt Requests in die passende Region. Ein regionaler Directory Cache findet die Home Cell. Dort pruefen lokale Identity- und Permission-Daten den Request, und nur Tenant-lokale Vector-Shards werden durchsucht.

Cells	Hunderte oder Tausende isolierte Einheiten mit automatischem Lifecycle.
Storage Tiers	Hot, warm und cold mit klaren Retrieval- und Retention-Regeln.
Global IDs	Zeitlich sortierbar, kollisionsfrei und ohne zentrale Sequenz.
Cost Control	Quoten, Budgets, Sampling und Abrechnung pro Tenant.
Blast Radius	Rollouts, Schemafehler und Lastspitzen duerfen nur wenige Cells treffen.

Eine einzelne globale Vector-Datenbank waere nicht automatisch das Ziel. Tenant-Daten sollen moeglichst lokal bleiben. Globale Suche ueber alle Organisationen waere fachlich meist falsch, sicherheitskritisch und extrem teuer.

56. Multi-Region Konsistenz nach Datenart

API-Key Revocation	Schnell und konsistent; Sicherheit bevorzugt Verfuegbarkeit.
Memory Writes	Tenant Home Region als Single Writer; kontrolliertes Failover.
Embeddings	Eventual Consistency ist akzeptabel; Recency-Fallback bleibt aktiv.
Claims	Single Writer pro Tenant und Claim-Schluessel.
Dashboard	Leicht veraltete Read Models sind meist akzeptabel.
Analytics	Asynchron und vom synchronen Request-Pfad entkoppelt.

CAP ist keine Auswahlbox fuer jede Datenbank. Es beschreibt Entscheidungen waehrend einer Netzwerkpartition. Energon trifft diese Entscheidung pro Datenart. Sicherheit und Konfliktentscheidungen brauchen staerkere Konsistenz; Embeddings und Metriken duerfen nachlaufen.

Active-active Writes fuer denselben Tenant erzeugen Konflikt-, Reihenfolge- und Auditkomplexitaet. Ein Tenant Home Region Modell ist oft die bessere erste globale Architektur. Erst echte Verfuegbarkeitsziele und gemessene Failover-Anforderungen rechtfertigen komplexere Replikation.

TEIL 8 / PROFESSIONELLES URTEIL

Die Codebase kritisch und fair beurteilen

Gute Ingenieure unterscheiden zwischen sauberer Grundlage, bewusster frueher Vereinfachung und einem Risiko, das vor der naechsten Skalierungsstufe geloest werden muss.

57. Was die Codebase bereits richtig macht

- Klare Crate-Grenzen zwischen Domain, Transport, Persistenz, Worker und SDK.
- Agent-Identitaet wird serverseitig aus einem Credential abgeleitet.
- Private Writes werden vor geteilter Promotion erzwungen.
- Permission Filtering findet vor Core-Ranking und Packing statt.
- PostgreSQL-Constraints bilden eine zweite Verteidigung.
- Embedding-Erzeugung ist vom Write-Request entkoppelt.
- Bei Embedding-Fehlern existiert ein funktionaler Fallback.
- Outbox und fachliche Mutation teilen dieselbe Transaktion.
- Events besitzen Schema-Versionen und deduplizierbare IDs.
- Claims trennen freie Notiz von strukturierter, konfliktfaehiger Aussage.
- Candidate Limits begrenzen Arbeit auf dem synchronen Request-Pfad.

Diese Entscheidungen sind wichtiger als exotische Infrastruktur. Sie schaffen Invarianten, an denen eine groessere Plattform spaeter weiterbauen kann.

58. Die wichtigsten Grenzen vor horizontaler Skalierung

IDs	Prozesslokale Zaehler durch UUIDv7 oder ULID ersetzen.
Idempotenz	Mutation Requests mit Idempotency Keys absichern.
Rate Limits	Globale Gateway-Grenzen plus lokale Schutzschicht.
Embeddings	Worker-Leases, Modellversion und echtes Chunking.
Retrieval	Semantic Score transportieren und Hybrid Ranking evaluieren.
Tokens	Modellspezifischen Tokenizer statt Wortheuristik verwenden.
Audit Inserts	Items in Batches statt pro Zeile schreiben.
Migrations	Dedizierter Deployment-Job statt jeder API-Replik.

Nicht jede Grenze ist heute ein Produktionsfehler. Prioritaet folgt dem naechsten realen Last- und Zuverlaessigkeitsziel. Eine Cell-Architektur zu bauen, bevor Idempotenz und Observability funktionieren, waere die falsche Reihenfolge.

59. Observability und SLOs

Ein SLO beschreibt ein messbares Zuverlaessigkeitsziel. Beispiel: 99,9 Prozent erfolgreicher Context Builds pro Monat, mit P95 unter 300 Millisekunden ohne externen Embedding-Cache-Miss. Das genaue Ziel muss zum Produkt passen.

API	Request Count, Status, Dauer, In-Flight und Body-Groesse.
Database	Pool Wait, Query Duration, Locks, Deadlocks und Replication Lag.
Retrieval	Candidate Count, Vector Duration, Fallbacks, Recall@K und Token-Nutzung.
Worker	Queue Age, Batch Rate, Provider Errors, Retries und Publish Acks.
Business	Context Builds, Memories, Promotions, Conflicts und Kosten pro Tenant.

Durchschnittswerte verbergen Ausreisser. P50 beschreibt den typischen Request, P95 und P99 die langsamen Ränder. Fehlerbudgets verbinden SLO und Entwicklungsgeschwindigkeit: Ist das Budget verbraucht, haben Zuverlaessigkeitsarbeiten Vorrang vor riskanten Features.

60. Teststrategie fuer Energon

Unit Tests	Permissions, Claims, Packing, Normalisierung und Backoff.
Integration	Echtes PostgreSQL, Migrationen, Constraints, Locks und pgvector.
Contract	SDK JSON, Header, Statuscodes und Versionskompatibilitaet.
Property Tests	Viele generierte Scopes und Identitaeten duerfen nie Tenant-Grenzen brechen.
Load Tests	Realistische Tenant-Verteilung, Bursts, Hot Tenants und Worker Lag.
Chaos	Provider-Ausfall, DB-Failover, NATS-Unterbrechung und Worker-Crash.

Security-Invarianten verdienen negative Tests. Nicht nur pruefen, dass der Besitzer Private Memory sieht, sondern systematisch, dass andere Agents, Rollen, Projekte, Organisationen und ungepruefte User-Claims sie niemals sehen.

Vector Retrieval braucht ein gelabeltes Eval-Set. Ohne relevante und irrelevante Query-Chunk-Paare kann man Recall, Precision, Thresholds und neue Embedding-Modelle nicht objektiv vergleichen.

61. Wie du jede neue Datei liest

Beginne nicht bei jedem Semikolon. Baue zuerst die Verantwortung der Datei auf. Danach lies Typen, Eingaben, Ausgaben und Fehlerpfade. Erst dann gehe Zeile fuer Zeile.

1. Schicht	Core, API, DB, Worker, SDK oder UI?
2. Eingang	Welche Daten kommen herein und welche davon sind untrusted?
3. Ausgang	Welcher Wert, Effekt oder Fehler verlaesst die Funktion?
4. Ownership	Was wird bewegt, geliehen, geklont oder gemeinsam geteilt?
5. Invariante	Welche Regel darf nach der Funktion niemals verletzt sein?
6. Failure	Was passiert bei Timeout, Crash, Duplikat oder Teilfehler?
7. Scale	Welche Arbeit ist begrenzt und welche kann mit Datenmenge wachsen?
8. Evidence	Welche Tests, Metriken oder Audits beweisen das Verhalten?

Senioritaet bedeutet nicht, jede Bibliotheksfunktion auswendig zu kennen. Sie bedeutet, Annahmen sichtbar zu machen, Grenzen zu erkennen, Risiken zu priorisieren und Erfolg messbar zu definieren.

62. Abschlussuebungen

Bearbeite diese Aufgaben ohne sofort in den Code zu schauen. Danach vergleiche deine Antwort mit der echten Implementierung.

- Zeichne den Weg eines Context Requests vom SDK bis zum Audit und markiere jede Trust Boundary.
- Erklaere, warum ein semantisch perfekter Treffer trotzdem nicht in den Context Pack gehoeren kann.
- Sage fuer jeden Parameter von MemoryRecord::from_write voraus, ob er bewegt, geliehen oder kopiert wird.
- Entwirf einen Idempotency-Key-Workflow fuer memory/write inklusive PostgreSQL-Constraint.
- Erklaere den Crash zwischen JetStream Ack und mark_published und mache den Consumer sicher.
- Entwirf eine Re-Embedding-Migration ohne Retrieval-Downtime.
- Berechne RPS, Concurrency und Datenwachstum fuer dein eigenes realistisches Agent-Verhalten.
- Entwirf Tenant Placement und Failover fuer drei Regionen mit unterschiedlichen Datenschutzregeln.
- Definiere drei SLOs und die Metriken, die sie beweisen.

Der letzte Merksatz

> Ein gutes System ist nicht das mit den meisten Komponenten. Es besitzt klare Invarianten, begrenzte Arbeit, explizite Fehlerzustaende, messbare Ziele und einen nachvollziehbaren Weg zur naechsten Groessenordnung.

Wenn du Ownership, Datenfluss, Trust Boundaries, Transaktionen, Retrieval-Scores, Backpressure und Failure Isolation an der Energon-Codebase erklaeren kannst, lernst du nicht nur Rust. Du lernst, Systeme zu bauen, die auch unter Last und Fehlern noch verstaendlich bleiben.